

Matematične Tehnologije

ANTONIO MONTERO

December 16, 2025

Contents

0.1	LaTeX	1
0.1.1	Uvod v LaTeX	2
0.1.2	Osnovna okolja	11
0.1.3	Sklicevanje in literatura	17
0.1.4	Matematika v LaTeXu	21
0.1.5	LaTeX: Razno	24
0.1.6	Beamer: Prestavitve v LaTeXu	31
0.2	GeoGebra	37
0.2.1	GeoGebra: Uvod	38
0.2.2	Dinamična geometrija z GeoGebro	51
0.2.3	Analiza z GeoGebro	53
0.2.4	Tridimenzionalna geometrija z GeoGebro	55
0.2.5	GeoGebra: Izvoz in ustvarjanje učnih gradiv	56
0.3	SageMath	57
0.3.1	SageMath: Uvod	58
0.3.2	Algebra s SageMath	67
0.3.3	Sage: Analiza in risanje grafov.	68
0.4	Priloge	69
0.4.1	Nameščanje LaTeXa na svoj računalnik	70
0.4.2	Izvoz datotek iz Overleaf projekta	72
0.4.3	Dolžine v LaTeXu	73

0.1 LaTeX

0.1.1 Uvod v LaTeX

Kaj je LaTeX?

LaTeX je program za oblikovanje besedil. LaTeX je nadgradnja programa TeX, ki je razvil Donald E. Knuth. leta 1997. Prvo verzijo LaTeX-a je napisal Leslie Lamport (1980). LaTeX je paket dodatnih ukazov za TeX, ki avtorjem omogočajo oblikovanje in izpisovanje dokumentov v najvišji kakovosti s pomočjo že pripravljenih profesionalnih vzorcev.

Trenutna standardna verzija je LaTeX2e (1994), zanj pa je odgovoren Frank Mittelbach. LaTeX se izgovori 'láteh', po angleško pa običajno rečemo 'lejteh'.

Ko želi *avtor* kaj objaviti, pošlje svoj rokopis založniku. Tam *knjižni opremljevalec* določi obliko dokumenta (širino stolpcev, obliko črk, velikost presledkov pred in po naslovih ...). Opremljevalec svoja navodila skupaj z rokopisom preda *črkostavcu*, ki na podlagi teh podatkov stavi knjigo. V LaTeX-u prevzame vlogi knjižnega oblikovalca in črkostavca, da se avtor osredotoči na vsebino Figure 1.

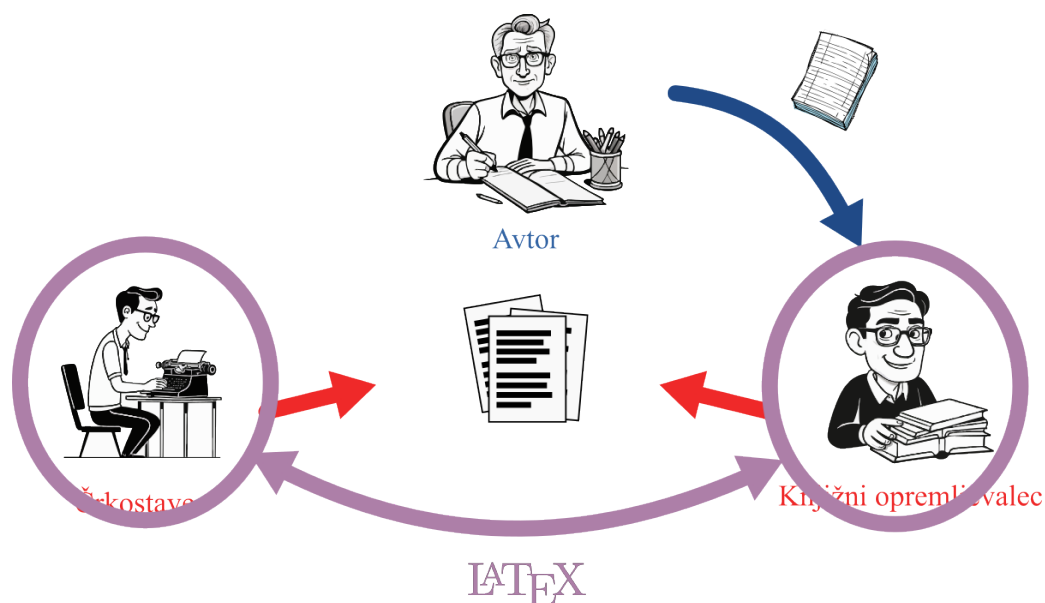


Figure 1: Kaj je LaTeX?

Pomebno: Filozofija LaTeXa

Oblikovanje dokumentov z LaTeXom sledi naslednjemu načelu:

mi (avtorji) se ukvarjamo z **vsebino** dokumenta, LaTeX pa poskrbi za **oblikovanje**.

Ali, enakovredno,

ne skrbimo za **oblikovanje**, zato se lahko osredotočimo na **vsebino**.

Kaj zmore LaTeX?

LaTeX zmore skoraj vse, kar zmorejo drugi urejevalniki besedil. Vendar pa je LaTeX posebej primeren za pisanje znanstvenih in tehničnih besedil, ki vsebujejo veliko matematičnih formul in simbolov. LaTeX omogoča tudi enostavno ustvarjanje tabel, grafov in drugih elementov, ki so pogosto potrebni v tehničnih besedilih. Poleg tega LaTeX omogoča enostavno ustvarjanje kazal, bibliografij in drugih elementov, ki so pomembni za znanstvena besedila.

Čeprav je LaTeX optimiziran za izdelavo znanstvenih in tehničnih besedil, ni omejen samo na to. Danes uporabljam LaTeX za pisanje vseh dokumentov, ki naj izgledajo profesionalno ali lepo. To sega od osebnih in poslovnih pisem do pesmarice za igranje kitare.

Vprašal sem moje sodelavce, kaj delajo z LaTeXom. Tukaj je nekaj zanimivih odgovorov:

- Knjige, članke, in zapiski predavanj;
- Predstavitve, s pomočjo Beamer-ja [Beamer: Predstavitve v LaTeXu](#);
- Profesionalna dokumentacija (npr CV, projekte vloge);
- Pogodbe za stanovanje;
- Pisma;
- Pravila za družabne igre
- Note (za glasbo)
- Facebook, SMS, Instagram (da, res je!).
- **Zaključna dela (diplome, magistrska, doktorat)**

Če vas zanima, si lahko tukaj ogledate mojo doktorsko disertacijo, ki je bila napisana izključno z uporabo programa LaTeX.

LaTeX ni Word

MS Word, Google Docs, LibreOffice, so WYSIWYG (kar vidiš, to tudi dobiš). V teh programih avtor obliko dokumenta določi interaktivno med samim vnosom besedila v računalnik. Med celotnim procesom na zaslonu vidi, kakšna bo končna oblika, ko bo dokument natisnjen.

Pri LaTeXu ni možno videti končne oblike med samim tipkanjem besedila. Lahko pa vidimo končno obliko na zaslonu, potem ko datoteko prevedemo z LaTeXom. Tako lahko popravke naredimo še preden dokument izpišemo s tiskalnikom. V urejevalnikih WYSIWYG strukturo in obliko urejate hkrati z pisanjem besedila. Zlasti pri dolgih projektih se je zelo lahko izgubiti. LaTeX sili avtorja, da določi logično strukturo besedila, LaTeX pa potem izbere najprimernejšo obliko.

Zakaj LaTeX? Predstavljajmo si, da pišete roman v svojem najljubšem urejevalniku WYSIWYG. V tej knjigi se odvijata dve vzporedni zgodbi v različnih dimenzijah. Da bi bralcu sporočili, katera od dimenzij je trenutno opisana, ste se odločili, da boste za obe uporabili različne barve. Vaša knjiga bi tako lahko izgledala takole:

„Ne, ne moreš me zapustiti!“ je zaklical Peredur Lancelotu.

„Moram,“ je odgovoril. „Usoda me kliče. Toda ponovno se bova srečala. Obljubim.“

Shao je občutila nepojasnjeno žalost, kot da bi pravkar izgubila nekaj ali nekoga, ki ji je bil pomemben.

„Si v redu?“ je vprašala Ashby, medtem ko je jedla svojo kašo.

„Ne izgledaš dobro.“

„V redu sem, samo utrujena.“

Ko ste končali prvi osnutek (ki obsega 300 strani), ste se odločili, da ga pošljete po e-pošti prijatelju, da vam bo povedal svoje mnenje. Vendar se je izkazalo, da lahko tiskalnik vašega prijatelja tiska samo črno-belo, zato ne more natisniti datoteke, ki ste mu jo poslali.

Enostavna rešitev! Vse zeleno besedilo spremenite v kursivno pisavo, ostalo pa pustite v pisavi upright front. Po ročni spremembi vsakega odstavka prvih 120 strani knjige ste se spomnili, da v nekaterih primerih za poudarjanje uporabljate kursivno pisavo: [Ali boš *to* jedla?]{.latex-line .text-green} Sedaj boste morali preveriti vsak odstavek za kurzivo in ga spremeniti v drug slog, preden spremenite vse zeleno besedilo v kurzivo.

Vidite neskončni problem?.

LaTeX rešuje ta problem tako, da avtorju omogoča, da se osredotoči na vsebino in logično strukturo besedila, medtem ko LaTeX poskrbi za oblikovanje.

Kako deluje LaTeX?

Klasična razlaga delovanja LaTeX-a je naslednja:

1. Avtor napiše besedilo in ukaze LaTeX v datoteko `.tex` (npr. `ime_datoteke.tex`).
2. Datoteka `.tex` prevedemo z LaTeX-om, ki ustvari datoteko `.dvi` (npr. `ime_datoteke.dvi`).
3. Datoteke `.dvi` niso standardne, zato je treba datoteka `.dvi` pogosto spremeniti v datoteko `.pdf`; na koncu dobimo datoteko `ime_datoteke.pdf`, ki jo lahko natisnemo ali delimo z drugimi.

V sodobnem času pogosto gremo iz `ime_datoteke.tex` datoteke neposredno v `ime_datoteke.pdf` npr. z pomočjo programa `pdflatex`.

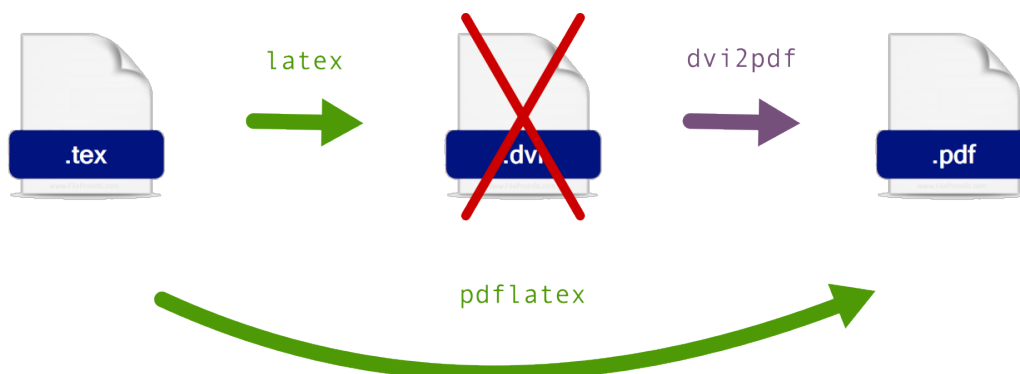


Figure 2: Kajo deluje LaTeX?

Kako začeti z LaTeXom?

Lahko namestite na svoj računalnik. Če vas to želite, pogledjte [Nameščanje LaTeXa na svoj računalnik](#).

Uporabili bomo drugačen pristop in delali z LaTeXom na spletu. To bomo storili prek Overleaf, ki je platforma za delo z LaTeX projekti na spletu. Najprej morate ustvariti račun.

Ukazi in okolja

Kot smo že omenili, se moramo pri delu z LaTeXom osredotočiti na vsebino, oblikovanje pa prepustiti LaTeXu. Ker pa je LaTeX “samo” računalniški program, potrebuje več vodenja. Avtor mora navesti dodatne informacije, ki opisujejo logično strukturo njegovega dela. Te informacije se navedejo prek *ukazov* in *okolij*, ki se vpišejo okoli besedila.

Ukazi v LaTeXu imajo splošno obliko:

```
\imeUkaza[neobvezni argument]{argument}
```

in se običajno uporablja za dajanje LaTeXu določenih navodil.

- Ukaz *vedno* se začne z znakom `\`.
- Argumentov je lahko več, npr. `{prvi}{drugi}...{zadnji}` ali nobenega (npr. ukaz `\tiny`, vendar je to redko).
- Lahko je več neobveznih argumentov: `[prvi, drugi, ..., zadnji]` ali noben (in potem ne napišemo ničesar). Pazite razliko z obveznimi argumenti.

Okolje je posebna vrsta ukaza z naslednjo strukturo:

```
\begin{okolje}
Vsebina okolja: ukazi in besedilo
\end{okolje}
```

Okolja običajno nadzorujejo, kako se bo del besedila/kode obnašal, ko bo natisnjen v dokument.

Zaenkrat vam ni treba skrbeti, kako delujejo ukazi in okolja. To bomo spoznavali sproti. V poglavju [Osnovna okolja](#) bomo podrobneje pogledali nekatera najpomembnejša okolja.

Table 1: Razredi LaTeX dokumentov

Razred	Opis
<code>article</code>	za članke v znanstvenih revijah, kratka poročila in druge krajše dokumente.
<code>proc</code>	razred za zbornike, ki temelji na razredu <i>article</i> .
<code>report</code>	za daljša poročila, ki vsebujejo več poglavij, manjše knjige, doktorske disertacije, ...
<code>book</code>	za prave knjige.
<code>beamer</code>	za predstavitve.
<code>letter</code>	za pisma.

Preambula in telo datoteke

Vsaka datoteka LaTeX ima naslednjo obliko:

```
%Preambula
\documentclass{...}
\usepackage{...}
%...
% Telo
\begin{document}
  Vsebina dokumenta.
\end{document}
```

Najpomembnejše okolje v lateksu je okolje `document`. To okolje se lahko uporabi največ enkrat na dokument in v tem okolju pa napišemo dejansko vsebino našega dokumenta. Vse, kar je znotraj tega okolja, se imenuje *telo* dokumenta.

Če je najpomembnejše okolje `document`, mora biti najpomembnejši ukaz `\documentclass`. Vse, kar je med ukazom `\documentclass` in začetkom okolja `document`, se imenuje *preambula* dokumenta. Tukaj nadzorujemo splošno delovanje LaTeXa.

Opozirilo!

Preambula običajno vsebuje samo ukaze (v nasprotju z običajnim besedilom) in obstajajo ukazi, ki se lahko uporabljajo samo v preambuli. Pogosta napaka v LaTeXu je, da se del kode napiše na napačno mesto.

Ukaz `\documentclass` določa vrsto dokumenta, ki ga želimo ustvariti. Uporabljamo ga tako

```
\documentclass[možnosti]{razred}
```

Tu `razred` določa vrsto dokumenta, ki ga želimo ustvariti (npr. članek, knjiga, poročilo ...), `možnosti` pa so dodatne nastavitve, ki vplivajo na videz dokumenta (npr. velikost pisave, velikost papirja ...). V Table 1 so prikazani različni razredi dokumentov v LaTeXu.

V tem učbeniku bomo obravnavali le razreda `article` in `beamer` (glej [Beamer: Predstavitve v LaTeXu](#)), vendar bo večio tega, kar se bomo naučili, mogoče brez težav uporabiti tudi v drugih razredih.

Nekatere od uporabnih `možnosti` so `10pt` (prim. `11pt`, `12pt`), `a4paper` (prim. `letterpaper`), `onecolumn` (prim. `twocolumn`). Privzete možnosti za `article` razred so [`letterpaper`, `10pt`, `oneside`, `onecolumn`, `final`].

Avtor, naslov in datum

Z ukazi `\title`, `\author` in `\date` lahko določimo naslov, avtorja in datum dokumenta. Ti ukazi se običajno uporabljajo v preambuli, vendar jih lahko uporabimo tudi v telesu dokumenta. Ti ukazi ne

ustvarijo ničesar, dokler ne uporabimo ukaza `\maketitle`, ki ustvari naslovno stran z informacijami, ki smo jih določili z zgornjimi ukazi. Poglejte Example ??.

Example 0.1.1. % Preamble

```
% ...
\author{France Prešeren}
\title{Zdravljica}
\date{26.04.1848}
% ...
% Telo
\begin{document}
\maketitle
V tem dokumentu bom napisal ljubezensko pismo Juliji.
\end{document}
```

Paketi in jeziki

LaTeX je zelo prilagodljiv in razširljiv sistem. Osnovni LaTeX ponuja le osnovne funkcije, vendar lahko z uporabo *paketov* dodamo nove funkcije in možnosti. LaTeX ima okoli 6000 paketov na voljo. Paketi so zbirke ukazov in okolij, ki jih lahko vključimo v naš dokument z ukazom

```
\usepackage{ime_paketa}.
```

Opozorilo!

Ukaz `\usepackage{ime_paketa}` mora biti v preambuli, torej pred `\begin{document}`.

Eden najpomembnejših paketov je `babel`, ki omogoča uporabo različnih jezikov. Paket naložimo z ukazom

```
\usepackage[jezik]{babel}
```

kjer `jezik` določa jezik, ki ga želimo uporabiti (npr. `slovene`, `english`, `spanish`, `french`, `german`, `italian`, ...). Paket `babel` poskrbi za pravilno oblikovanje besedila glede na izbrani jezik (npr. vezaje, ločila, datum, ...).

Opomba!

- V starih različicah LaTeXa nismo mogli preprosto pisati č, š, ž, á, ü ...
- Te črke lahko vedno dobimo z ukazi, kot so:
- Alternativno lahko uporabimo naslednje pakete za kodiranje znakov:

```
% Vhodnega dokumenta,
\usepackage[utf8x]{inputenc}
% opcije lahko so [utf8], [utf8x], ali [cp1215]

% izhodnega dokumenta,
\usepackage[T1]{fontenc}
% lahko damo tudi [T2A] za cirilica
```

- Sodobni LaTeX običajno ne potrebuje teh paketov. Ponavadi je dovolj če uporabimo paket `babel`.

Table 2: Posebni znaki v LaTeXu

Znak	Ukaz
%	\%
\$	\\$
&	\&
#	\#
_	_
{ }	\{ \} (vedno v paru)
~	\textasciitilde
^	\textasciicircum
\	\textbackslash

Komentarji, posebni znaki, prazne vrstice in presledki.

Nasvet!

Pri delu v LaTeXu besedila **ne brišemo**, temveč ga **komentiramo**.

Če želimo izpizati simbol %, zapišemo \%. Simbol % je v LaTeXu poseben znak, to pomeni, da ima poseben pomen in ga ne moremo uporabiti kar tako. Drugi posebni znaki v LaTeXu so prikazni v Table 2.

Pogosta napaka!

Pogosta napaka v LaTeXu je, da se pozabi uporabiti ukaz za enega od posebnih simbolov.

Vežaj vs pomišljaj

Vežaj v LaTeXu dobimo z enim samim znakom -. Pogoste uporabe vežaja so:

- sestavljene besede, npr. `informacijsko-komunikacijska tehnologija`: `[informacijsko-komunikacijska tehnologija]{.latex-line}`;
- sklanjanje kratic, npr. v NUK-u: v NUK-u.

Pomišljaj v LaTeXu dobimo z dvema znakoma -. Pogoste uporabe pomišljaja so:

- za označevanje intervalov, npr. `strani 10-20`: strani 10—20;
- ločevanje besedila, npr. za pojasnila v stavku, npr. `LaTeX - program za oblikovanje besedil` - je zelo prilagodljiv: `LaTeX — program za oblikovanje besedil` — je zelo prilagodljiv.

Narekovaji

LaTeX ima posebna pravila za tiskanje narekovajev:

- Enojne narekovaje dobimo z uporabo znakov ‘ in ’, npr. `'narekovaj'`: ‘narekovaj’.
- Dvojne narekovaje dobimo z uporabo znakov " in ", npr. `"narekovaji"`: “narekovaji”.
- Za slovenske narekovaje uporabimo ukaza `\glqq` in `\grqq`, npr. `\glqq slovenski narekovaji\grqq`: „slovenski narekovaji“.
- Če uporabimo `\usepackage[slovene]{babel}`, lahko preprosto napišemo “ in ”, npr. `"slovenski narekovaji"`: „slovenski narekovaji“.
- Lahko uporabimo tudi "> in "<, npr. `>slovenski narekovaji<`: »slovenski narekovaji« (potreben je tudi paket `babel`).

Pogosta napaka!

Pogosta napaka v LaTeXu je, da uporabljamo naravne narekovaje (" in ') ali pa napačno uporabljamo ukaze za narekovaje, npr.

- "narekovaji"
- "narekovaji"

Presledki

- Poljubno število presledkov (ali tabulatorjev) med besedami se obravnava kot en sam presledek.
- Presledek na začetku vrstice se ignorira.
- Nova vrstica se obravnava kot presledek, če ni prazna.
- Prazna vrstica se obravnava kot konec odstavka in začne nov odstavek.
- Zaporedne prazne vrstice se obravnavajo kot ena sama prazna vrstica.
- Prazen komentar % ni prazna vrstica.

Example 0.1.2.

Nasvet!

Ker se nova vrstica v LaTeXu obravnava kot presledek, razen če je prazna, je dobro, da vsak stavek napišete v eno vrstico. To olajša urejanje besedila in sledenje spremembam v sistemih za nadzor različic, kot je Git.

Včasih resnično potrebujemo prelom vrstice, ne da bi začeli nov odstavek. Za to imamo na voljo naslednji dve ukazi.

- Ukaz `\` ustvari prelom vrstice.
- Ukaz `\newline` ustvari prelom vrstice in ignorira vse presledke na začetku naslednje vrstice.
- Ukaz `\linebreak[arg]` (*arg* je število od 0 do 4, privzeta vrednost je 4.): 0 pomeni, da je prelom vrstice nizke prioritete in se lahko, zaradi drugih okoliščin, ignorira, 4 pa pomeni, da je prioritete preloma vrstice visoka.
- Ukaz `\noindent` prepreči zamik prve vrstice novega odstavka.

Example 0.1.3.

LaTeX ima tudi ukaze za obravnavanje prelomov strani.

- Ukaz `\pagebreak[arg]` je najbolj primitivni med njimi. Preprosto prekine stran na mestu, kjer se pojavi. Argument *arg* deluje enako kot pri `\linebreak`.
- Ukaz `\newpage` prekine stran in začne novo stran. Prav tako zapolni preostali prostor na strani s praznim prostorom.
- Ukaz `\clearpage` deluje podobno kot `\newpage`, vendar poleg tega tudi izpiše vse plavajoče elemente (npr. slike in tabele), ki še niso bili izpisani.
- Ukaz `\cleardoublepage` deluje podobno kot `\clearpage`, vendar zagotavlja, da se nova stran začne na desni strani v dvostranskem tisku.

Nevarnost!

Ukazi za prelome vrstic in strani so namenjeni uporabi v izjemno specifičnih okoliščinah, vendar **njihova uporaba ni priporočljiva**.

Ne pozabite, da moramo oblikovanje dokumenta prepustiti LaTeXu.

Pisave in velikosti pisave

LaTeX privzeto uporablja pisavo *Computer Modern*, ki jo je zasnoval Donald Knuth. Sodobne različice LaTeX pogosto uporabljajo Latin Modern, ki je sodobna različica Computern Modern. Če ta pisava ni privzeto naložena, jo lahko (in jo tudi morate) naložiti s ukazom `\usepackage{lmodern}` v preambuli.

LaTeX samodejno izbere pisavo in velikost črk glede na razred dokumenta in izbrane pakete. Če želite spremeniti pisavo ali velikost pisave, LaTeX uporablja štirje parametri:

family (družina) Dejanska pisava, ki jo uporablja. Privzeto ima LaTeX tri družine: *roman* oz. *pokončna serifna* pisava (`\textrm`), *sans serif* oz. *gladka* pisava (`\textsf`) in *strojepisna* oz. *enakoprostorna* pisava (`\texttt`).

series (serija) Debelina pisave. Privzeto ima LaTeX dve seriji: *medium* oz. *srednja* pisava (`\textmd`) in *bold face* oz. *kreпка* pisava (`\textbf`).

shape (oblika): Naklon pisave. Privzeto ima LaTeX tri oblike: *italic* oz. *poševna* pisava (`\textit`), *slanted* oz. *ležeča* pisava (`\textsl`) in *small caps* oz. *male velike črke* (`\textsc`). Poleg tega sta na voljo dve virtualni obliki: *pokončna* (`\textup`) in *velika-mala* (`\textulc`). To v resnici nista obliki, ampak uporabniški ukazi. Prvi preklopi nazaj na pokončno pisavo, drugi pa onemogoči male velike črke. Ukaz `\textnormal` je le kombinacija obeh.

size (velikost). To se večinoma nadzira z možnostmi ukaza `\documentclass`, vendar o tem več kasneje.

Če želite spremeniti enega od prvih treh parametrov v danem besedilu, preprosto uporabite ukaze, kot je `\ukaz{dano besedilo}`. Npr. z ukazom `\textbf{To je krepko besedilo}` dobimo **To je krepko besedilo**`{.latex-line}` in z ukazom `\textit{To je poševno besedilo}` dobimo *To je poševno besedilo*`{.latex-line}`.

Pazite!

Lahko združimo več ukazov, prikazanih zgoraj, (npr. `\textit{\textbf{To je krepko in poševno besedilo}}`) proizvaja **To je krepko in poševno besedilo**`{.latex-line}`; vendar niso vse kombinacije na voljo. LaTeX bo pritoževal, če boste poskusili uporabiti kombinacijo, ki ne obstaja.

Vsi zgornji ukazi imajo različico *switch*. Namesto da bi prejeli besedilo prek argumenta, trajno spremenijo pisavo, dokler se ta ponovno ne spremeni. V Table ?? so prikazani vsi ukazi in njihove različice *switch*.

Example 0.1.4.

Poleg zgoraj navedenih ukazov lahko ustvarimo tudi podčrtano besedilo (`\underline{besedilo}`) in poudarjeno besedilo (`\emph{besedilo}`). Delovanje ukaza `\emph{besedilo}` je odvisno od konteksta. Običajno ustvari besedilo, ki je zelo podobno besedilu, ustvarjenemu z ukazom `\textit`, vendar ni namenjeno za pisanje poševna pisava. Poglejte Example ?? za primer.

Example 0.1.5.

Za razliko od večine urejevalnikov besedil WYSIWYG (Word, itd.) LaTeX nima izrecnega načina za določanje velikosti pisave. Dejanska velikost besedila je odvisna od možnosti argumenta `\documentclass` (10pt, 11pt ali 12pt). To pomeni, da bo navadno besedilo velikosti 10pt (oz. 11pt ali 12pt). Velikost pisave za druge dele besedila bo določena ustrezno.

Če resnično želimo spremeniti velikost pisave za določen del besedila, uporabimo ukaze v Table ?. Natančneje, ti ukazi niso LaTeX ukazi, ampak TeX ukazi (to pomeni, da niso sodobni ukazi). Zaradi tega je njihova sintaksa nekoliko drugačna. Delujejo kot `{\velikost <besedilo>}`, kjer *velikost* je eden od ukazov iz tabele in *besedilo* je besedilo, ki ga želimo spremeniti. Npr. z ukazom `{\Huge To je največje besedilo, ki ga lahko ustvari LaTeX.}` dobimo stavek:

To je največje besedilo, ki ga lahko ustvari LaTeX.

V Table ?? podajamo dejansko velikost za vsako od njih ter relativno primerjavo med različnimi velikostmi.

Mnoge od teh ukazov imajo svojo sodobno različico v obliki okolja. Namesto da napišete `{\small majhna pisava}`, lahko na primer napišete `\begin{small} majhna pisava \end{small}`.

Opozorilo!

To je res pomembno, zato je vredno ponoviti: Preveč igranja s pisavami in velikostmi pisav je v nasprotju s filozofijo LaTeXa. Ne pozabite, da moramo LaTeXu prepustiti oblikovanje, medtem ko se **mi osredotočimo na vsebino**.

Vaje

Vsako poglavje v tej knjigi se konča s povzetkom različnih vaj ter dodatnimi vajami ali domačimi Vajami. Tukaj je ustrezna vaja za prvo poglavje:

- Exercise ??: Ustvarjanje računa v Overleafu.
- Exercise ??: Ustvarjanje prve LaTeX datoteke.
- Exercise ??: Dodajanje avtorja, naslova in datuma.
- Exercise ??: Uporaba paketa `babel`.
- Exercise ??: Uporaba komentarjev in posebnih znakov.
- Exercise ??: Uporaba posebnih znakov.
- Exercise ??: Uporaba vezajev, pomišljajev in narekovajev.
- Exercise ??: Uporaba različnih pisav.
- Exercise ??: Uporaba različnih velikosti pisav.

0.1.2 Osnovna okolja

Opomba

Včasih, ko vadimo z LaTeXom, se znajdemo v situaciji, ko moramo vstaviti lažno besedilo, tj. besedilo, katerega vsebina ni pomembna. LaTeX ima za to poseben paket, in sicer `lipsum`. Ta paket nam omogoča dostop do 150 odstavkov besedila Lorem Ipsum, ki je delček besedila “*De finibus bonorum et malorum*” avtorja Cicero.

Ko pri vsakem paketu, ga morate naložiti z ukazom `\usepackage{lipsum}` v preambuli. Nato lahko vstavite lažno besedilo z ukazom `\lipsum[<razpon odstavkov>][<razpon stavkov>]`, kjer sta `<razpon odstavkov>` in `<razpon stavkov>` razpon odstavkov in stavkov, ki jih želimo vstaviti.

Na primer, `\lipsum[1-3]` bo vstavilo prve tri odstavke besedila Lorem Ipsum, medtem ko bo `\lipsum[4][1]` vstavilo samo prvi stavek četrtega odstavka.

Logična struktura dokumenta

LaTeX ima na voljo posebni ukazi za logično strukturo dokumenta. Sintaksa teh ukazov je `\ime_enote{naslov}`, kjer je `ime_enote` ime loginčne enote, npr. `section`, medtem ko je `naslov` besedilo naslova, ki ga želimo prikazati v dokumentu.

Dejanske logične enote, ki so na voljo, so odvisne od razreda dokumenta, npr. za razred `article` so na voljo enote `section`, `subsection`, `subsubsection`, `paragraph` in `supargraph`.

Razreda `report` in `book` imata na voljo tudi ukazi `\chapter{naslov}`.

Example 0.1.6.

Vse te razrede dokumentov imajo tudi enote `part`, ki ga lahko uporabite za razdelitev dokumenta na več delov, ne da bi to vplivalo na oštevilčenje poglavij, razdelkov itd.

Razmik med odstavki, oštevilčenje in velikost pisave naslovov bo samodejno nastavil LaTeX. To pomeni, da vam ni treba ročno oštevilčevati naslovov in da se, če na primer, odločite, da boste dodali novo poglavje na začetek dokumenta, vsa oštevilčenje poglavij, razdelkov itd. se bo samodejno posodobilo.

Ukaz `\tableofcontents` natisne kazalo našega dokumenta, ki sledi strukturi, ki smo jo določili z zgoraj opisanimi ukazi. Upoštevajte, da je treba ta ukaz uporabiti v telesu datoteke, da se kazalo natisne na ustrezno mesto.

Vsi ukazi za logično strukturo dokumenta imajo tudi “*z zvezdico*” različico, npr. `\section*{naslov}`, ki ustvari naslov brez oštevilčenja in ne vključuje naslova v kazalo.

Včasih je naslov našega razdelka zelo dolg in ne želimo, da se tako natisne v kazalu. V tem primeru lahko uporabimo naslednjo sintakso, da natisnemo krajšo različico naslova.

```
\section[Kratki naslov za kazalo]{Dolgi in še posebno dologočasen naslov,
ki se izpiše na začetku razdelka
ampak noro bi bilo,
če bi se tako izpisal tudi v kazalu}
```

Poravnava besedila

LaTeX privzeto poravnava besedilo z obeh strani, kar pomeni, da prilagodi prostor med besedami, tako da so vse vrstice približno enake dolžine. V urejevalnikih WYSIWYG se ta nastavek pogosto imenuje *obojestransko poravnano*.

Včasih želimo besedilo poravnati drugače, zato ima LaTeX tri različne okolja: `flushleft`, `flushright` in `center`. Vendar je izvedba tega okolja nekoliko zastarela in povzroča težave z razdeljevanjem besed (kar LaTeX običajno opravi sam). Sodobne različice teh okolij so bile implementirane v paketu `ragged2e`. Ta paket morate naložiti v preambuli z ukazom `\usepackage{ragged2e}`. Ta paket ponuja okolja `FlushLeft`, `FlushRight` in `Center`, ki so izboljšane različice zgoraj omenjenih okolij.

Njegova uporaba je preprosta in lažje razumljiva s pomočjo vaje.

Seznami

LaTeX ima tri vrste seznamov: *urejene seznane*, *neurejene seznane* in *opise*. Sintaksa za vse tri vrste seznamov je podobna. Začnemo z okoljem, ki ga želimo uporabiti, nato pa vsak element seznama začnemo z ukazom `\item`. Za urejene seznane uporabimo okolje `enumerate`, za neurejene seznane okolje `itemize`, za opise pa okolje `description`.

Example 0.1.7.

Podobno kot pri logičnih enotah, LaTeX samodejno izvede oštevilčenje urejenih seznamov. Lahko tudi gnezditi seznane, tj. imeti seznam znotraj seznama.

Example 0.1.8.

Paket `enumitem` (v ang.), nam omogoča prilagoditev videza seznamov. Na primer, lahko spremenimo oznake neurejenih seznamov ali prilagodimo zamik elementov seznama. Za uporabo tega paketa ga moramo naložiti v preambuli z ukazom `\usepackage{enumitem}`. Ko smo naložili paket, lahko uporabimo seznam lastnosti po začetku ustreznega okolja.

Možnosti uporabe tega paketa so ogromne in vključujejo prilagajanje oznak, boljši razmik med seznami, seznane v vrstici, različne sloge za opisne seznane in drugo. Tukaj bomo obravnavali le nekaj osnovnih primerov.

Example 0.1.9.

Example 0.1.10.

Seveda, ročno nastavljanje lastnosti vsakega seznama je nevarno in v nasprotju s filozofijo LaTeXa. Globalne lastnosti lahko nastavimo na naslednji način.

Example 0.1.11.

Opozorilo!

Dejstvo, da lahko seznane močno prilagodimo, še ne pomeni, da bi to morali storiti. Na splošno bi morali izbrati slog in se ga držati. Poleg tega je privzeti slog pogosto dovolj dober za mnoge namene.

Tabele

Tabele v LaTeXu so bile obravnavane na več načinov. LaTeX ima zelo primitivni način obravnavanja navpično poravnanih vsebin, ki jih je mogoče enostavno pretvoriti v tabelo. Obstaja več paketov, ki razširjajo to funkcionalnost. Eden od prvih in najpogostejše uporabljenih je paket `array`. Ta paket morate običajno naložiti, ko delate s tabelami.

Glej tudi

Obstaja veliko paketov, ki na različne načine razširjajo paket `array`. Vsi niso medsebojno združljivi, zato njihovo podrobno raziskovanje presega obseg tega učbenika. Vendar pa so najpogostejše uporabljane funkcionalnosti za tabele zajete v enem od naslednjih dveh virov (v angleščini):

- Tabele — Dokumentacija Overleaf.
- LaTeX wikibooks — tabele.

LaTeX uporablja okolje `tabular` za tiskanje vsebine, ki je poravnana tako navpično kot vodoravno, torej nekaj, kar je zelo podobno tabeli. Sintaksa prva vrstica okolja je

```
\begin{tabular}[pos]{<poravnava stolpcev>}
```

Table 3: Specifikacija stolpca za okolje `tabular`

Znak	Opis
<code>l</code>	Poravnava besedila v stolpcu na levo.
<code>c</code>	Poravnava besedila v stolpcu na sredino.
<code>r</code>	Poravnava besedila v stolpcu na desno.
<code>p{<širina>}</code>	Besedilo v stolpcu bo zavito, če preseže določeno širino.
<code>m{<širina>}</code>	Kot <code>p{<širina>}</code> , vendar bo besedilo navpično poravnano na sredino celice.
<code>b{<širina>}</code>	Kot <code>p{<širina>}</code> , vendar bo besedilo navpično poravnano na dno celice.

kjer je `pos` neobvezna možnost, ki določa navpično poravnavo tabele glede na preostali del besedila (privzeto je `c`, tj. središče, Vendar sta možna tudi `b` (za dno) in `t` (za vrh)). Argument `<poravnava stolpcev>` je niz, ki določajo številko in poravnavo stolpcev. Možnosti za ta niz so pojasnjene v Table 3. Nekatere od njih lahko zahtevajo paket `array`.

Example 0.1.12.**Example 0.1.13.****Example 0.1.14.**

Za ločevanje stolpcev v tabeli uporabimo znak `&`, za začetek nove vrstice pa ukaz `\\`. Če želimo navpično črto med stolpci, uporabimo znak `|` v nizu `<poravnava stolpcev>`. Za vodoravne črte, ki se raztezajo čez celotno širino tabele, uporabimo ukaz `\hline` na koncu vrstice. Za vodoravne črte, ki se raztezajo čez določene stolpce, uporabimo ukaz `\cline{i-j}`, kjer `i` in `j` določata začetni in končni stolpec, čez katerega se črta razteza.

Example 0.1.15.

Zgoraj opisani ukazi so že dovolj za izdelavo kompleksnih tabel v LaTeXu. Vendar pa končni rezultat ni nujno lep in izgleda nekoliko zastarel.

Obstaja veliko paketov, ki razširjajo funkcionalnosti okolja `tabular`. Osredotočili se bomo na paket `booktabs`, ki je zasnovan za izdelavo tabel v LaTeXu, ki so primerne za objavo. Ta paket velja za sodoben standard za izdelavo tabel v LaTeXu.

Ta paket ustvarja tabele v skladu z naslednjima dvema konvencijama:

1. Lepe tabele ne potrebujejo navpičnih črt.
2. Lepe tabele ne zlorablajo vodoravnih črt, zlasti pa nikoli ne smemo uporabljati dvojnih vodoravnih črt.

Vse, kar je izrecno obravnavano v tej knjigi, je združljivo s paketom `booktabs`, vendar lahko nekateri drugi klasični pristopi (zlasti navpične črte na tabelah) povzročijo nepričakovane rezultate.

Najprej moramo naložiti paket z ukazom `\usepackage{booktabs}` v preambuli. Nato lahko uporabimo ukaze `\toprule`, `\midrule` in `\bottomrule` za ustvarjanje vodoravnih črt na vrhu, sredini in dnu tabele.

Example 0.1.16.

Simbol `@{sep}` lahko uporabimo za vstavljanje vsebine *med stolpce*. Tukaj je `sep` niz, ki bo vstavljen med celice ustreznih stolpcev. Ta sintaksa je uporabna za vstavljanje dodatnega prostora med stolpci z uporabo ukaza `\hspace{<širina>}` kot vrednost za niz `sep`. Lahko jo uporabimo tudi za vstavljanje posebnih znakov. Naslednji primer naj pojasni ta koncept.

Example 0.1.17.

Ukaz `\multicolumn{n}{<poravnava>}{besedilo}` združi n stolpcev v en stolpec, ki vsebuje `besedilo` in je poravnan glede na `<poravnava>`. Ta ukaz je uporaben, kadar želimo, da se besedilo razteza čez več stolpcev, na primer za naslove stolpcev.

Delno vodoravno črto iz stolpca i v stolpec j natisnemo z ukazom `\cmidrule{i-j}`. To je podobno ukazu `\cline`, ki smo ga preučili prej, vendar je del paketa `booktabs`. Črta, ki jo dobimo, je tanjša in izgleda bolj estetsko.

Včasih potrebujemo, da se celica razteza čez več vrstic (v nasprotju z več stolpci). To je mogoče v LaTeXu, ko naložimo paket `multirow`. Ta paket nam omogoča uporabo ukaza `\multirow`, ki ima naslednjo sintakso.

```
\multirow[np]{<štVrstic>}{<širina>}[nGibanje]{besedilo}
```

kjer je `np` navpična poravnava celice glede na preostali del besedila (privzeto je `c`, tj. središče, vendar sta možna tudi `b` (za dno) in `t` (za vrh)). Argument `štVrstic` je število vrstic, čez katere se celica razteza, `širina` pa širina celice (lahko uporabimo `*`, da LaTeX sam določi širino ali `=` za enako kot stolpec, v tem primeru, da je stolpec določen z `p{<širina>}`, `m{<širina>}` ali `b{<širina>}`). Neobvezni argument `nGibanje` določa navpično premikanje besedila v celici (privzeto je `0pt`, kar pomeni, da je besedilo navpično poravnano na sredino celice). Nazadnje je `besedilo` vsebina celice.

Za razliko od ukaza `\multicolumn`, ukaz `\multirow` ne nadomesti celic v vrsticah, ki jih pokriva. To pomeni, da moramo v teh vrsticah pustiti prazne celice (tj. uporabiti `&` za ločevanje stolpcev, vendar brez vsebine med njimi).

Ko je treba stolpec oblikovati na določen način, je neprijetno v vsako celico vstaviti iste ukaze. Poleg tega, če se odločite, da je treba nekaj spremeniti v oblikovanju, bi morali vsako celico urediti posebej. Da bi se temu izognili, paket `array` opredeljuje `>{<ukazi>}` in `<{<ukazi>}` specifikacije stolpcev, ki se lahko uporabijo za vstavljanje kode pred in za stolpec.

Example 0.1.18.

Čeprav je uporaba ukazov `>{...}` in `<{...}` priročna, če imamo samo en tak stolpec, postane hitro neprijetna, ko se v kateri koli tabeli pojavi več stolpcev istega tipa. V tem primeru je morda zaželeno uporabiti ukaz

```
\newcolumnntype{<ime>}{<definicija>}
```

za definiranje nove vrste stolpca, ki ga lahko uporabimo v okolju `tabular`. Tukaj je `<ime>` ime nove vrste stolpca (ena črka), `<definicija>` pa je definicija vrste stolpca, ki lahko vključuje ukaze `>{...}` in `<{...}`.

Example 0.1.19.**Glej tudi!**

Ukazi, opisani v tem razdelku, so dovolj za izdelavo lepih tabel v LaTeXu. Vendar pa obstaja veliko paketov, ki razširjajo funkcionalnosti okolja `tabular`. Če bralca zanimajo bolj napredne funkcije, naj si ogleda naslednje vire (v angleščini):

- Paket `longtable`. za tabele, ki se raztezajo čez več strani: [dokumentacija]
- Paket `colortbl` za barvne tabele.
- Paket `tabularx` za tabele, ki se prilagajajo širini strani.

Slike

LaTeX sam po sebi ne podpira vključevanja slik, vendar pa to omogoča paket `graphicx`. Ta paket moramo naložiti v preambuli z ukazom `\usepackage{graphicx}`. Nato lahko uporabimo ukaz `\includegraphics` za vključevanje slik v dokument. Ta ukaz ima naslednjo sintakso.

```
\includegraphics[<lastnosti>]{<pot do slike>}
```

kjer je `<pot do slike>` pot do datoteke s sliko (relativna ali absolutna). Datoteka slike je lahko v različnih formatih, vendar so najbolj podprti formati `.png`, `.jpeg` in `.pdf`. Lahko izpustite končnico datoteke in LaTeX bo poskusil naslednje (v tem vrstnem redu): `.pdf`, `.png`, `.jpg`, `.mps`, `.jpeg`, `.jbig2`, `.jb2`.

Neobvezni argument `<lastnosti>` je seznam lastnosti, ki jih želimo uporabiti pri vključitvi slike. Najpogosteje uporabljena lastnost sta `width` in `height`, ki določata širino oz. višino slike. Če samo ena od teh lastnosti določi, bo druga samodejno prilagojena, da ohrani razmerje stranic slike.

Example 0.1.20.

Namesto ročnega določanja širine ali višine slike lahko uporabimo tudi možnosti `scale` za povečanje ali pomanjšanje slike za določen faktor. Prenos negativnih vrednosti bo zrcalil sliko.

Example 0.1.21.

Če želimo sliko zavrteti, lahko uporabimo možnost `angle`, ki določa kot vrtenja v stopinjah (v nasprotni smeri urinega kazalca). Privzeto je slika zavrtana okoli svojega spodnjega levega kota, vendar lahko uporabimo možnost `origin`, da spremenimo to točko. Možne vrednosti za `origin` so `c` (središče), `l` (levo), `r` (desno), `t` (zgoraj), `b` (spodaj) in kombinacije teh vrednosti, kot so `lt` (levo zgoraj), `rb` (desno spodaj) itd.

Example 0.1.22.

Plavajoči elementi

Če poskusite napisati daljši tekst z več tabelami in slikami, boste kmalu opazili problem: vključitev teh elementov v običajni tekstovni tok bo povzročila veliko praznega prostora.

Poglejmo naslednji primer:

Example 0.1.23.

V prvi strani je veliko praznega prostora, ki ga povzroča slika. Da bi se temu izognili, LaTeX ponuja koncept plavajočih elementov, ki vključuje slike in tabele. Plavajoči elementi so lahko postavljeni na najbolj primerno mesto v dokumentu, ne da bi motili besedilo.

LaTeX ponuja dve okolji za plavajoče elemente: `figure` za slike in `table` za tabele. Ta okolja imajo podobno sintakso.

```
\begin{figure}[<pozicija>]
%...Vsebina slike, običajno z ukazom \includegraphics...
\end{figure}
```

```
\begin{table}[<pozicija>]
%... Vsebina tabele, običajno z okoljem tabular...
\end{table}
```

kjer je `<pozicija>` neobvezna možnost, ki določa, kje naj LaTeX poskuša postaviti plavajoči element. Možne vrednosti so v Table 4.

Na primer, `\begin{table}[!hbp]` pomeni, da naj LaTeX poskuša postaviti tabelo tukaj, če je mogoče (`h`), sicer na dno strani (`b`), če to ni mogoče, pa na posebno stran samo za plavajoče elemente (`p`); to vse pa lahko naredi tudi, če rezultat ni najlepši možen (`!`). Če ne podamo nobene možnosti, LaTeX privzeto uporabi `tbp`.

Z ukazom `\caption{<pojasnilo>}` lahko dodamo pojasnilo plavajočemu elementu. Ta ukaz mora biti znotraj okolja `figure` ali `table`. Oštevilčenje in niz "Table" oziroma "Figure" se dodata samodejno. Seveda, če smo v preambuli spremenili jezik dokumenta z ukazom `\usepackage[<jezik>]{babel}`, se bosta spremenila tudi ta niza. Običajno uporabljamo ukaz `\centering` znotraj okolja `figure` ali `table`, da centriramo vsebino.

Table 4: Pozicije plavajočih elementov

Oznaka	Objekt lahko stoji...
h	tukaj, na mestu, kjer je definiran v besedilu (samo za majhne elemente).
t	na vrhu strani.
b	na dnu strani.
p	na posebni strani, ki vsebuje samo plavajoče elemente.
!	brez upoštevanja večine vgrajenih parametrov, ki bi lahko preprečile, da je objekt postavljen na to mesto.

Example 0.1.24.

S ukazi `\listoftables` in `\listoffigures` lahko ustvarimo seznam tabel oziroma seznam slik, podobno kot z ukazom `\tableofcontents` za kazalo vsebine. Če uporabljamo dolga pojasnila, lahko podobno kot pri naslovih logičnih enot kot dodatni argument zapišemo kratko pojasnilo, ki se bo pojavilo v seznamu tabel oziroma seznamu slik:

```
\caption[<kratko pojasnilo>]{<dolgo pojasnilo>}
```

Včasih želimo vključiti več slik skupaj. To lahko storimo z uporabo paketa `subcaption`. Ta paket nam omogoča uporabo okolja `subfigure` znotraj okolja `figure`. Vsaka podslika lahko ima svoje pojasnilo z ukazom `\caption{<pojasnilo>}`. Celotna figura lahko ima tudi svoje pojasnilo z ukazom `\caption{<pojasnilo>}` znotraj okolja `figure`. Okolje `subfigure` ima podobno sintakso kot okolje `figure`, vendar mora imeti tudi obvezni argument, ki določa širino podslike.

Example 0.1.25.

Ukaz `\centering` znotraj okolja `figure` obravnava obe sliki kot eno samo sliko. Na splošno je dobro uporabiti relativne dolžine za razporeditev slik. V tem primeru določamo, da je vsaka podslika 40 % širine strani. Ukaz `\hfill` med podslikama vstavi vodoravni prostor, ki se razteza, da zapolni prostor med njima. Namesto `\hfill` lahko uporabimo tudi `\hspace{<širina>}`, da vstavimo fiksno širino prostora med podslikama. Podnaslovi se ne prikažejo pri tiskanju seznama slik (z uporabo ukaza `\listoffigures`), temveč le napisi, povezani s sliko. Okolje `subfigure` ima tudi neobvezni argument, ki določa pozicijo podslike glede na slike okoli nje; množne vrednosti so `t`, `b` in `c` (za vrh, dno, sredino).

Podobno kot podslike lahko ustvarimo tudi podtabele z uporabo okolja `subtable` znotraj okolja `table`. Uporaba je enaka kot pri podslikah, ampak ni priporočljiva. Za razliko od slik, skoraj ni primerov, ko bi imeli dve tabeli pod istim naslovom boljšo berljivost kot pod različnimi naslovi.

Vaje

- Exercise ??: Ustvarite dokument z več razdelki in podrazdelki.
- Exercise ??: Ustvarite tabelo z različnimi poravnavamami stolpcev in vrstic.
- Exercise ??: Ustvarite različne vrste seznamov (oštevilčene, neoštevilčene, opisne).
- Exercise ??: Ustvarite tabelo s paketom `booktabs`.
- Exercise ??: Uredite tabelo iz prejšnje vaje z uporabo ukaza `\multicolumn`.
- Exercise ??: Uredite tabelo iz prejšnje vaje z uporabo ukaza `\cmidrule`.
- Exercise ??: Uredite tabelo iz prejšnje vaje z uporabo ukaza `\multirow`.
- Exercise ??: Ustvarite dokument z vsaj tremi tabelami in dvema slikama kot plavajoče elemente.

0.1.3 Sklicevanje in literatura

Z LaTeXom lahko oblikujemo ne le dokumente, ki so visoke kakovosti za tiskanje. Vendar pa je danes veliko dokumentov prebrati na zaslonu računalnika ali mobilne naprave. Paket `hyperref` omogoča ustvarjanje dokumentov PDF z aktivnimi povezavami oz. *hiperpovezavami*. To pomeni, da lahko kliknemo na sklic in nas ta pripelje do ustreznega dela dokumenta ali na zunanjo spletno stran. Paket `hyperref` omogoča tudi ustvarjanje kazal, ki so interaktivni in omogočajo enostavno navigacijo po dokumentu.

Za zdaj bomo uporabili paket `hyperref` za sklicevanje na literaturo v našem dokumentu. Seveda, najprej moramo naložiti paket v preambuli našega dokumenta:

```
\usepackage{hyperref}
```

Pazite!

Paket `hyperref` mora biti naložen kot zadnji paket v preambuli, saj lahko povzroči konflikte z drugimi paketi, če ni naložen na koncu.

Sklicevanje

V dokumentu lahko sklicujemo na različne elemente, kot so slike, tabele, enačbe, razdelki, itd. Za sklicevanje uporabimo ukaza `\label{<oznaka>}` in `\ref{<oznaka>}`. Ukaz `\label{<oznaka>}` postavimo takoj za element, na katere se želimo sklicevati. Ukaz `\ref{<oznaka>}` pa uporabimo na mestu, kjer želimo vstaviti sklic. Argument `<oznaka>` je poljuben niz znakov, ki ga izberemo sami, vendar mora biti edinstven v celotnem dokumentu.

Example 0.1.26.

Nasvet

Oznake je priporočljivo oblikovati tako, da so opisne ampak kratke. Na primer, za razdelek o sklicevanju, `sec:sklicevanje`, je boljša oznaka kot `s1` ali `razdelek1`.

Dobra praksa je tudi, da uporabljamo predpone za različne vrste elementov, kot so `sec:` za razdelke, `fig:` za slike, `tab:` za tabele in `eq:` za enačbe.

Opomba

Ukaz `\ref` vrne le številko elementa, na primer številko razdelka ali slike. Potrebno je, da ročno dodamo besedilo, kot je "Razdelek", "Slika" ali "Tabela", pred številko.

Obstajajo paketi, kot je `cleveref`, ki omogočajo samodejno dodajanje besedila pred številko. Na žalost, paket `cleveref` ni podpira slovenskega jezika, in čeprav je mogoče definirati nove jezike, je trenutno slovenski jezik težko implementirati zaradi sklanjatev (prim. „glej Sliko 1" v „na Sliki 1").

Če uporabimo ukaz `\ref{<oznaka>}` z razdelki "z zvezdico" (npr. `\section*{...}`), ki niso oštevilčeni, bo ukaz vrnil nič ali nepričakovano vrednost, saj ti razdelki nimajo številke. Zato je priporočljivo, da se na razdelke "z zvezdico" ne sklicujemo z ukazom `\ref`. Možna alternativa je uporaba ukaza `\nameref{<oznaka>}`, ki vrne ime razdelka.

Ukaz `\pageref{<oznaka>}` vrne številko strani, na kateri se nahaja element z dano oznako. Ta ukaz lahko uporabimo z razdelki "z zvezdico", saj nam ni treba pridobiti številke razdelka, ampak le številko strani.

Za plavajoče elemente, kot so slike in tabele, je priporočljivo, da oznako dodamo znotraj okolja `figure` ali `table`, takoj po ukazu `\caption{...}`. To zagotavlja, da se oznaka nanaša na pravilno številko slike ali tabele.

Example 0.1.27.

Označujemo in sklicujemo lahko vse, čemur LaTeX dodeli številko. Na primer, točke v seznamih.

Example 0.1.28.**Literartura****Klasični način**

Za vključitev literature v dokument LaTeX, lahko uporabimo okolje `thebibliography`. To je le poseben seznam, kjer vsaka referenca dobi svojo oznako z ukazom `\bibitem{<oznaka>}`. Nato se na referenco sklicujemo z ukazom `\cite{<oznaka>}`.

Example 0.1.29.

Ta način je enostaven in hiter, vendar ima nekaj pomanjkljivosti:

- Ročno moramo vnašati vse podatke o referencah, kar je lahko zamudno in nagnjeno k napakam.
- Ni standardiziranega formata za vnos referenc, kar lahko vodi do neenotnosti v slogu citiranja.
- Enostavne spremembe sloga citiranja so težavne, saj moramo ročno prilagoditi vsako referenco.
- Ni ločnega/struturnega označevanja, ki bi nakazovalo, kateri del bibliografskega vnosa je avtor, naslov, leto izdaje itd.
- Sortiranje referenc je ročno in lahko vodi do napak. Urejanje (npr. po letu izdaje namesto po priimku avtorja) je prav tako ročno in zamudno.
- Na splošno to ni pristop, ki bi bil značilen za LaTeX.

Zaradi teh razlogov je priporočljivo uporabiti bolj napredne metode za upravljanje literature v LaTeX dokumentih, kot je BibTeX ali/in BibLaTeX.

BibLaTeX način

BibTeX je orodje za upravljanje bibliografskih podatkov, ki deluje skupaj z LaTeXom. Omogoča nam, da shranimo vse naše reference v ločeno datoteko `.bib`, kar olajša upravljanje in ponovno uporabo referenc v različnih dokumentih. BibLaTeX je sodobnejša alternativa BibTeX-u, ki ponuja več funkcionalnosti in prilagodljivosti.

Uporaba BibLaTeX-a vključuje naslednje korake:

1. Ustvarimo datoteko z bibliografskimi podatki z razširitvijo `.bib`. V tej datoteki so reference shranjene v posebnem formatu.
2. V preambulo našega LaTeX dokumenta vključimo paket `biblatex` in določimo slog citiranja ter datoteko z bibliografskimi podatki.

Datoteka `.bib` je sestavljena iz vnosa, kjer vsak vnos predstavlja eno referenco. Vsak vnos ima svoj edinstven ključ, ki ga uporabimo za sklicevanje na to referenco v našem dokumentu. Vsak vnos vsebuje različna polja, kot so avtor, naslov, leto izdaje, itd. in izgledajo nekako takole:

```
@<vrsta_Vnosa>{<ključ>,
<polje1>      = {vrednost1},
<polje2>      = {vrednost2},
<polje3>      = {vrednost3},
<poljeN>      = {vrednostN},
}
```

kjer je `<vrsta_Vnosa>` lahko `book`, `article`, `inproceedings`, `misc`, itd., odvisno od vrste reference. `<ključ>` je edinstven identifikator za ta vnos, ki ga bomo uporabili v ukazu `\cite{<ključ>}` v našem LaTeX dokumentu. `<polje1>`, `<polje2>`, itd. so različna polja, ki vsebujejo informacije o referenci, kot so avtor, naslov, leto izdaje, itd. Npr. za knjigo *Harry Potter in kamen modrosti* bi bil vnos videti takole:

```
@book{hp1,
author    = {J. K. Rowling},
title     = {Harry Potter in kamen modrosti},
publisher = {Založba EPTA},
year      = {1997}
}
```

Polja so odvisna od vrste vnosa in slog bibliografije, ki ga uporabljamo (več o tem kasneje). Za knjige so običajna polja `author`, `title`, `publisher` in `year`. Za članke v revijah bi lahko uporabili polja, kot so `journal`, `volume`, `number` in `pages`. Za spletne vire je pogosto uporabno polje `url` in `note`. Celoten seznam vrst vnosov in polj najdete v dokumentaciji paketa BibLaTeX ali tem članku na wikibooks.

Dobra novica je, da običajno ni potrebno ročno pisati datoteke `.bib`, saj obstajajo številna orodja in spletne strani, ki lahko samodejno generirajo vnose v pravilnem formatu. Nekatera orodja, ki jih lahko uporabite za ustvarjanje in upravljanje datotek `.bib`, vključujejo:

- Google Scholar: omogoča izvoz posameznih referenc v format `.bib`.
- COBISS: slovenski sistem za iskanje in izposajo knjižničnih gradiv, ki omogoča izvoz bibliografskih podatkov v format `.bib`.
- ZoteroBib: spletno orodje za hitro ustvarjanje bibliografij, ki podpira izvoz v format `.bib`.
- MathSciNet: spletna baza podatkov za matematiko.
- DBLP: baza podatkov za računalništvo.

Ko smo pripravili našo bazo podatkov referenc (tj. našo datoteko `.bib`, recimo `literatura.bib`), moramo v našem LaTeX dokumentu naložiti paket `biblatex` in povezati datoteko z bibliografskimi podatki. To storimo z naslednjimi ukazi v preambuli:

```
\usepackage{biblatex} % Naloži paket biblatex
\addbibresource{literatura.bib} % Poveže datoteko z bibliografskimi podatki, recimo literatura
```

V besedilu našega dokumenta se na reference sklicujemo z ukazom `\cite{<ključ>}`, kjer je `<ključ>` ključ vnosa v naši datoteki `.bib`. Parameter `<ključ>` lahko je tudi seznam ključev, ločenih z vejico, če želimo sklicevati na več referenc hkrati.

Na koncu dokumenta (ali, kjerkoli že želimo natisniti seznam referenc), dodamo ukaz `\printbibliography`, ki natisne seznam vseh referenc, na katere smo se sklicevali v besedilu. Upoštevajte, da čeprav nismo uporabili vseh referenc iz datoteke `.bib`, bo `\printbibliography` natisnil le tiste, na katere smo se sklicevali z ukazom `\cite{<ključ>}`.

Z uporabo BibLaTeX lahko na nadzorovan način prilagodimo videz naših citatov in sklicev. To storimo z izbiro sloga citiranja, ki ga določimo kot možnost pri nalaganju paketa `biblatex`. Na primer,

```
\usepackage[style=alphabetic]{biblatex}
```

naloži slog `alphabetic`, ki uporablja črke iz imen avtorjev in leto izdaje za označevanje citatov v besedilu. Privzeto je slog `numeric`, ki uporablja številke za označevanje citatov. Obstaja veliko različnih slogov citiranja, ki jih lahko uporabimo, odvisno od naših potreb in preferenc. Obsežen seznam slogov najdete v dokumentaciji paketa BibLaTeX ali na Overleaf.

Pazite!

Slog citatov lahko spreminjamo neodvisno, to pomeni, da lahko spreminjamo, kaj ukaz `\cite` natisne v našem besedilu in kako se natisnejo oznake, ko natisnemo bibliografijo, vendar je dobro ohraniti enak slog.

Če damo možnost `sorting=ynt` razvrsti bibliografijo po letu izdaje (`y`), nato po imenu avtorja (`n`) in nato po naslovu (`t`). Privzeto je `sorting=nty`, kar pomeni, da se najprej razvrsti po imenu avtorja, nato po naslovu in nato po letu izdaje.

Table 5: Naslov bibliografije v kazalu.

Vrednost	Opis
<code>bibliography</code>	privzeto, kot je razdelek »z zvezdico«.
<code>bibintoc</code>	kot razdelek z zvezdico, vendar se bo pojavil v kazalu.
<code>subbibliography</code>	bo bibliografija padla za eno stopnjo v hierarhiji (<code>\section*</code> namesto <code>\chapter*</code> , <code>\subsection*</code> namesto <code>\section*</code> in tako naprej).
<code>bibnumbered</code>	naraven razdelek (brez zvezdice), torej s številko in v kazalu.
<code>subbibintoc</code>	podobno kot <code>subbibliography</code> , vendar bo prikazana v kazalu.
<code>subbibnumbered</code>	podobno kot <code>subbibliography</code> , vendar brez zvezdice.
<code>none</code>	ne bo natisnil naslova.

Celoten seznam možnosti za razvrščanje najdete v dokumentaciji paketa `BibLaTeX` ali na `BibLaTeX` cheat sheet.

Kot smo že omenili, ukaz `\printbibliography` natisne seznam vseh referenc, na katere smo se sklicevali v besedilu. Če želimo natisniti reference, na katere se nismo sklicevali, lahko uporabimo ukaz `\nocite{<ključ>}`. Ta ukaz nam omogoča, da vključimo določene vnose v bibliografijo, tudi če se nanje nismo sklicevali v besedilu. Če želimo vključiti vse vnose iz naše datoteke `.bib`, lahko uporabimo ukaz `\nocite{*}`.

Ukaz `\printbibliography` lahko sprejme različne možnosti, ki nadzorujejo videz bibliografije. Na primer, lahko uporabimo možnost `title`, da spremenimo naslov bibliografije, kot je prikazano spodaj:

Example 0.1.30.

Referenca lahko organizirate z uporabo filtrov. Na primer, lahko natisnemo samo knjige z uporabo možnosti `type=book`:

Example 0.1.31.

Na voljo so filtri `type`, `keywords` in `category` in lahko uporabimo tudi njihove negacije. Z ukazom `\defbibfilter{<ime_filtr>}` lahko definiramo tudi svoje filtre.

Example 0.1.32.

Example 0.1.33.

Privzeto se bibliografija ne prikaže v kazalu vsebine. Če želimo, da se prikaže, lahko uporabimo možnost `heading`. Različne vrednosti za možnost `heading` so v Table 5.

Example 0.1.34.

Example 0.1.35.

Example 0.1.36.

Example 0.1.37.

Example 0.1.38.

Example 0.1.39.

Example 0.1.40.

Vaje

- Exercise ??: Sklicevanje z uporabo paketa `hyperref`.
- Exercise ??: Sklicevanje na različne elemente v dokumentu.
- Exercise ??: Uporaba `BibLaTeX`-a za upravljanje literature.

0.1.4 Matematika v LaTeXu

LaTeX je še posebej primeren za pisanje matematičnih izrazov in ena izmed njegovih glavnih prednosti je prav podpora za matematične simbole in formule. LaTeX je že sam po sebi zelo zmogljiv. Vendar pa obstaja tudi veliko paketov, ki še dodatno razširijo njegove zmogljivosti na področju matematike.

Standardni paket za matematiko so:

- **amsmath**: paket, ki ga je razvila Ameriška matematična družba (American Mathematical Society) in ponuja številne izboljšave za pisanje matematičnih izrazov.
- **amssymb**: paket, ki ponuja dodatne matematične simbole.
- **amsfonts**: paket, ki omogoča uporabo dodatnih matematičnih pisav.
- **amsthm**: paket za definiranje in oblikovanje matematičnih izrekov, trditve, definicij itd.

Ti paketi so pogosto vključeni v predloge dokumentov, vendar jih je mogoče vključiti tudi ročno z ukazom `\usepackage{amsmath, amssymb, amsfonts, amsthm}` v preambuli vašega LaTeX dokumenta.

V sodoben LaTeX dokument je priporočljivo vključiti paket **mathtools**, ki je razširitev paketa **amsmath** in ponuja dodatne funkcije in izboljšave za pisanje matematičnih izrazov.

Pomembno!

V tem poglavju bomo predpostavljali, da so ti paketi že vključeni v vaš dokument. Lahko jih vključite z naslednjim ukazom v preambuli vašega LaTeX dokumenta:

```
\usepackage{mathtools} % mathtools vključuje tudi amsmath
\usepackage{amssymb, amsfonts, amsthm}
```

Osnovni matematični izrazi

LaTeX omogoča pisanje matematičnih izrazov na dva načina: znotraj besedila (*vrstični način*; ang. *inline mode*) in v ločenih vrsticah (*prikazen način*; ang. *display mode*).

Matematika v vrstičnem načinu se vstavlja z uporabo znakov `$...$` ali `\(...\)`. Na primer, izraz za Pitagorov izrek lahko zapišemo kot `$a^2 + b^2 = c^2$` (ali `\(a^2 + b^2 = c^2\)`), ki je natisnjen kot $a^2 + b^2 = c^2$.

Za prikaz matematičnih izrazov v ločenih vrsticah (in osredotočeno poravnano) se uporabljajo znaki `\[...\]` ali okolje `equation*` (sta enakovredna). Na primer, če je $ax^2 + bx + c = 0$ kvadratna enačba, potem je rešitev te enačbe:

```
\[x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2} \]
```

kar je natisnjeno kot

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}. \quad (1)$$

Ta izraz v vrstičnem načinu bi bil zapisan kot `$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}$`, kar je natisnjeno kot $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2}$.

Opomba

Matematika v vrstičnem načinu zapisana z `$...$` včasih se šteje za zastarelo. Priporočljivo je uporabljati `\(...\)`. Vendar, v praksi večinoma avtorjev uporabijo `$...$`, ker je hitreje za tipkanje in dobi se enak rezultat.

Podobno je tudi pri prikaznem načinu, kjer je priporočljivo uporabljati `\[...\]` (ali okolja `equation*`) namesto `$$ ... $$`. Vendar je `$$...$$` resnično zastarelo in **se ne sme uporabljati**.

Matematične izraze v prikaznem načinu je mogoče tudi oštevilčiti z uporabo okolja `equation` (brez zvezdice). Rezultat je podoben kot zgoraj, vendar bo enačba oštevilčena. Seveda, lahko sklicujemo na to enačbo v besedilu z uporabo ukazov `\label{<oznaka>}` in `\ref{<oznaka>}`:

Example 0.1.41.

Za enačbe lahko tudi uporabimo ukaze `\eqref{<oznaka>}` in `\tag{<opis>}`, ki omogočata bolj prilagojeno sklicevanje in označevanje enačb:

Example 0.1.42.

Example 0.1.43.

Ukazi in okolja za pisanje matematičnih aktivirajo matematični način, kjer so nekateri znaki in ukazi drugačni kot v običajnem besedilnem načinu. Na primer, presledki v matematičnem načinu nimajo pomena, medtem ko so v besedilnem načinu pomembni:

Vse črke so v matematičnem načinu obravnavane kot spremenljivke in so zato natisnjene v poševni pisavi (*italics*). Razmik med črkami je večji kot v običajnem besedilu.

Zaradi zgoraj navedenih razlogov moramo matematično besedilo **vedno** zapisovati v matematičnem načinu (v vrstici ali prikazano).

Če želimo v matematičnem načinu uporabiti običajno besedilo, lahko uporabimo ukaz `\text{...}`: why?

Stavljenje matematičnih formul

Možnosti za matematične simbole je ogromno, zato bomo prikazali le nekaj primerov. Seznam primerov tukaj je namenjen prikazu nekaterih najpogostejših primerov uporabe, vendar nikakor ni izčrpen.

Pri sestavljanju matematičnih formul vam bodo morda v pomoč naslednji viri.

- Comprehensive LaTeX Symbol List: Obsežen seznam LaTeX simbolov, ki vključuje simbole iz različnih paketov **Skoraj 500 strani**.
- Detexify: Orodje, ki vam omogoča, da narišete simbol, ki ga iščete, in orodje vam bo predlagalo ustrezne LaTeX ukaze za ta simbol.
- [Seznam osnovnih matematičnih simbolov](#)`{target="_blank"}`: Kratek seznam pogosto uporabljenih LaTeX matematičnih simbolov.

Izreki, dokazi, definicije, itd..

Ko sestavljanju matematičnih besedil je pogosto potrebno vključiti izreke, dokaze, definicije in podobne strukture (ki jih bomo zaradi enostavnosti imenovali "teoremi"). Paket `amsthm`, ponuja enostaven način za ustvarjanje teh struktur z uporabo okolij.

Najprej moramo definirati nova okolja za naše teoreme v preambuli dokumenta. To storimo z uporabo ukaza

```
\newtheorem{<ime_okolja>}[<drugi_teorem>]{<Napis>}[<razdelek>]
```

kjer je `<ime_okolja>` ime novega okolja in `<Napis>` je besedilo, ki se bo prikazalo pred številko teorema. Neobvezna argumenta `<drugi_teorem>` in `<razdelek>` določata, kako se bodo teoremi številčili (in lahko uporabimo samo ena od njih ali nobene). Uporabimo `<drugi_teorem>`, če želimo, da se novi teoremi številčijo skupaj z obstoječim teoremom, kjer je `<drugi_teorem>` ime okolja. Če želimo, da se številčenje teoremov ponastavi v vsakem razdelku, uporabimo `<razdelek>`, kjer je `<razdelek>` ime števca razdelkov (npr. `section` ali `subsection`). Če ne uporabimo nobenega od teh dveh argumentov, se bodo teoremi številčili neodvisno skozi celoten dokument (1, 2, 3, ...). Ukaz `\newtheorem*{<ime_okolja>}{<Naslov>}` definira različico okolja, ki ne bo imela številke.

Po definiciji okolja lahko uporabimo novo okolje v našem dokumentu z uporabo standardnega LaTeX okolja sintakse:

```
\begin{<ime_okolja>}[<Naslov>]
Vsebina teorema.
\end{<ime_okolja>}
```

Neobvezni argument `<Naslov>` omogoča dodajanje naslova teorema, ki bo prikazan poleg številke teorema.

Example 0.1.44.

V LaTeXu obstajajo tri različne sloge teoremov:

- `plain`: privzeti slog, krepko naslov in poševno telo.
- `definition`: krepko naslov in pokončno telo.
- `remark`: poševno naslov in pokončno telo.

Za nastavitve sloga teorema uporabimo ukaz

```
\theoremstyle{<slog>}
```

pred definicijo okolja teorema. Vsak teorem, ki je definiran po tem ukazu, bo uporabil določen slog.

Example 0.1.45.

Paket `amsthm` ponuja tudi okolje `proof` za pisanje dokazov. Okolje samodejno doda besedo “Dokaz” na začetek in simbol kvadrata na konec dokaza.

Example 0.1.46.

Ukaz `\qedhere` lahko uporabimo znotraj okolja `proof`, da postavimo simbol kvadrata na določeno mesto, običajno na konec enačbe.

Example 0.1.47.

Vaje

- Exercise ??: Napišite LaTeX kodo za osnovne matematične izraze.
- Exercise ??: Napišite LaTeX kodo za matematične izraze z uporabo okolji `align` in `align*` in `aligned`.
- Exercise ??: Napišite LaTeX kodo, da reproducira Maxwelllove enačbe.
- Exercise ??: Napišite LaTeX kodo za izreke in dokaze.

0.1.5 LaTeX: Razno

V tej poglavju so zbrani različni nasveti in triki za delo z LaTeX-om, ki ne sodijo v druge poglavja.

Obsežnejši projekti

Za obsežnejše projekte, kot so diplomske naloge, je priporočljivo razdeliti dokument na več datotek in jih vključiti v glavno datoteko z ukazom `\input{}` ali `\include{}`. To omogoča lažje upravljanje in urejanje posameznih delov dokumenta.

Ukaz `\include{<pot do datoteke>}` vključi vsebino navedene datoteke in samodejno začne novo stran pred in po vključitvi.

Ukaz `\includeonly{<ime_datoteke1>, <ime_datoteke2>, ...}` omogoča vključitev samo določenih datotek med kompilacijo, kar je uporabno za hitrejše pregledovanje določenih delov dokumenta.

Pazite!

- Ukaz `\include{<ime_datoteke>}` lahko uporabljamo samo v **telesu glavne datoteke** ne znotraj drugih vključenih datotek.
- Ukaz `\includeonly{<ime_datoteke1>, <ime_datoteke2>, ...}` mora biti postavljen v preambulo glavne datoteke. V telesu dokumenta morajo biti ukaze `\include{}` za vse datoteke, ki jih želimo vključiti.
- Ukaz `\include{}` vedno začne novo stran, zato ni primeren za vključevanje manjših delov besedila. Poleg tega, ta ukaz ne deluje znotraj okolij, kot so `\begin{figure}...\end{figure}` ali `\begin{table}...\end{table}`.

Ukaz `\include` je malo omejevalen in ima posebno uporabo: vključiti več poglavje (`chapter`) v velik projekt. Ta ukaz vedno začne novo stran, to je koristno ko uporabljamo ukaz `\includeonly{}`, ker se prelomi strani ne bodo premaknili, če bomo vključili samo določene datoteke.

Včasih je bolj smiselno uporabiti ukaz `\input{<pot do datoteke>}`, ki vključi vsebino navedene datoteke brez preloma strani. Ukaz `\input{}` lahko uporabljamo kjerkoli v dokumentu, tudi znotraj okolij, kot so `\begin{figure}...\end{figure}` ali `\begin{table}...\end{table}`.

Ukaz `\input{}` lahko uporabimo tudi za vključitev datotek z ukazi LaTeX-a, kot so definicije makrov ali nastavitve paketov. Pogosta uporaba je vključitev datoteke, recimo, `preambula.tex`, ki vsebuje vse nastavitve in pakete, ki jih želimo uporabiti v našem dokumentu.

Pazite!

Čeprav lahko uporabimo ukaz `\input` za naložitev paketov v našo datoteko, mora vrstica, ki vsebuje ukaz `\documentclass`, **vedno** biti v glavni datoteki, pred vsemi ukazi `\input`.

Barve

Za uporabo barv v LaTeX dokumentih uporabimo paket `xcolor`. Paket omogoča uporabo treh ukazov za barvanje besedila ali ozadij:

- `\textcolor[<model>]{<barva>}{<besedilo>}`: Barva besedilo z določeno barvo.
- `\color[<model>]{<barva>}`: Spremeni barvo besedila od tega mesta naprej. To je različica *switch* ukaza `\textcolor`, kot je ukaz `\bfseries` za krepko pisavo (prim. `\textbf{}`).
- `\mathcolor[<model>]{<barva>}{<matematični_izraz>}`: Barva matematični izraz z določeno barvo. Ne pozabite, da ukaze za *navadno* besedilo običajno ne delujejo v matematičnem načinu.

Za zdaj se ne skrbite za neobvezni parameter `<model>`. Ta parameter nadzira, kaj se pričakuje za argument `<barva>`. Če ni določen, se privzeto uporablja model `named`, ki omogoča uporabo vnaprej določenih imen barv, to pomeni, barve pokličemo z imeni.

	red
	green
	blue
	cyan
	magenta
	yellow
	black
	gray
	white
	darkgray
	lightgray
	brown
	lime
	olive
	orange
	pink
	purple
	teal
	violet

Barve, ki so vnaprej določene v paketu `xcolor`, so prikazane v Figure 3.

Seznam določenih barv lahko razširimo z uporabo možnosti, ki jih ponuja paket `xcolor`. Npr z možnostjo `dvipsnames` lahko uporabimo dodatne barve prikazane v Figure 4.

 Apricot	 Aquamarine	 Bittersweet	 Black
 Blue	 BlueGreen	 BlueViolet	 BrickRed
 Brown	 BurntOrange	 CadetBlue	 CarnationPink
 Cerulean	 CornflowerBlue	 Cyan	 Dandelion
 DarkOrchid	 Emerald	 ForestGreen	 Fuchsia
 Goldenrod	 Gray	 Green	 GreenYellow
 JungleGreen	 Lavender	 LimeGreen	 Magenta
 Mahogany	 Maroon	 Melon	 MidnightBlue
 Mulberry	 NavyBlue	 OliveGreen	 Orange
 OrangeRed	 Orchid	 Peach	 Periwinkle
 PineGreen	 Plum	 ProcessBlue	 Purple
 RawSienna	 Red	 RedOrange	 RedViolet
 Rhodamine	 RoyalBlue	 RoyalPurple	 RubineRed
 Salmon	 SeaGreen	 Sepia	 SkyBlue
 SpringGreen	 Tan	 TealBlue	 Thistle
 Turquoise	 Violet	 VioletRed	 White
 WildStrawberry	 Yellow	 YellowGreen	 YellowOrange

Figure 4: Dodatne barve, ki jih določa paket `xcolor` z možnostjo `dvipsnames`.

Druge možnosti so `svgnames`, `x11names`. Možnost `svgnames` določa 151 barv, ki so definirane v SVG standardu, medtem ko možnost `x11names` določa 317 barv, ki so bile prvotno definirane za X11 Window System. Celoten seznam barv je na voljo v dokumentaciji paketa `xcolor`.

Drugi način pridobivanja novih barv je z mešanjem dveh ali več barv. Sintaksa za mešanje dveh barv je naslednja:

```
<barva1>!<odstotno>!<barva2>
```

Rezultat je barva, ki je sestavljena iz `<odstotno>` odstotkov `<barva1>` in $(100 - \text{odstotno})$ odstotkov `<barva2>`. Na primer, `red!40!blue` ustvari vijolično barvo, ki je mešanica 40% rdeče in 60% modre.

Mešanje lahko vključuje tudi več kot dve barvi. V tem primeru, mešanje je izvedeno zaporedno. Na primer, `red!50!green!30!blue` najprej zmeša 50% rdeče in 50% zelene, nato pa rezultat zmeša z 70% modre in 30% prej dobljene barve.

Včasih moramo biti natančnejši pri barvah, ki jih uporabljamo. Za to lahko uporabimo izbirni argument `model`. Najlažje ga razumemo prek modela `Gray`. V tem modelu določimo odtenek sive barve z vrednostjo med 0 in 15, kjer 0 predstavlja črno in 15 belo.

Podobno lahko uporabimo model `grey`, kjer vrednosti segajo od 0 do 1, pri čemer 0 predstavlja črno in 1 belo.

V Table ?? so prikazani nekateri barvni modeli, ki jih podpira paket `xcolor`.

Opozorilo!

Pri mešanju barv v različnih modelih lahko dobimo nepričakovane rezultate. Na primer, da izrazi `<barva1>!25!<barva2>` in `<barva2>!75!<barva1>` ne dajeta vedno iste barve, če sta `<barva1>` in `<barva2>` definirani v različnih barvnih modelih. Zato je priporočljivo, da pri mešanju barv vedno uporabljamo barve iz istega modela. Alternativno v poglavje 6 dokumentacije paketa `xcolor` najdemo več informacij o spremenjenju med barvnimi modeli.

Z uporabo ukaza `\definecolor{ime_barve}{<model>}{<parametri>}` (v preambuli) lahko definiramo svoje barve. Na primer, z ukazom `\definecolor{ULrdeca}{RGB}{226,25,30}` definiramo barvo z imenom `ULrdeca`, ki je rdeča barva Univerze v Ljubljani.

Doslej smo obravnavali ukaze za barvanje besedila. Možno je tudi spremeniti barbo ozadja strani z ukazom

`\pagecolor[<model>]{<barva>}`.

Ta ukaz je različica *switch*, to pomeni da spremeni barvo celotne strani od mesta, kjer je uporabljen, naprej. Če želimo vrniti nazaj privzeto barvo ozadja (prozoren), uporabimo ukaz `\nopagecolor`.

V izrazu za barvo lahko uporabimo znak `-` (minus), da dobimo nasprotno barvo. Na primer, `-white` predstavlja črno barvo.

Če želimo spremeniti barvo samo določenega območja besedila, lahko uporabimo ukaz `\colorbox[<model>]{<barva>}{<besedilo>}`. Ta ukaz obdaja besedilo z ozadjem določene barve. Ukaz `\fcolorbox[<model>]{<barva_okvirja>}{<barva_ozadja>}{<besedilo>}` obdaja besedilo z okvirjem določene barve in ozadjem določene barve.

Ukazi in okolja po meri.

LaTeX omogoča ustvarjanje lastnih ukazov in okolij, kar lahko olajša pisanje ponavljajočih se struktur v dokumentu. Klasični način za definiranje novega ukaza je z uporabo ukaza `\newcommand[<ime_ukaza>][<število_argumentov>]{<definicija>}` in novega okolja z uporabo ukaza `\newenvironment{<ime_okolja>}[<število_argumentov>]{<začetek_definicije>}{<konec_definicije>}`. Čeprav ti ukazi delujejo povsem normalno za osnovne definicije novih ukazov in okolij, bodo postopoma postali zastareli. Tega pristopa tukaj ne bomo obravnavali, vendar če bralca zanima, priporočamo branje Overleafovih navodil za nove ukaze `{target=_blank}` in nova okolja `{target=_blank}` (v angleščini).

Sodobnejši in bolj zmogljiv način za definiranje novih ukazov in okolij je z uporabo ukaza `\NewDocumentCommand` in `\NewDocumentEnvironment`. Ta pristop omogoča bolj fleksibilno določanje argumentov in podpira različne vrste argumentov, kot so obvezni, neobvezni, in več vrst argumentov.

Ukaz `\NewDocumentCommand` ima naslednjo sintakso:

`\NewDocumentCommand{<ime_ukaza>}{<vrsta_argumentov>}{<definicija>}`

- `<ime_ukaza>`: Ime novega ukaza, ki ga želimo definirati (vključno z `\`).
- `<vrsta_argumentov>`: Niz, ki določa vrste argumentov, ki jih ukaz sprejema. To bomo počasi obravnavali v nadaljevanju.
- `<definicija>`: Telo ukaza, kjer lahko uporabimo argumente znotraj ukaza.

Najlažje ukaz, da lahko definiramo je brez argumentov, to pomeni z praznim nizom za `<vrsta_argumentov>`.

Ta način je zelo uporaben, ko obstajajo del besedila ali strukture, ki jih pogosto uporabljamo v dokumentu. Seveda, če se kdajkoli odločimo spremeniti to besedilo ali strukturo, moramo to storiti samo na enem mestu: v definiciji ukaza.

Pogosta uporaba novih ukazov je pri definiciji matematičnih izrazov, ki jih pogosto uporabljamo. Kot smo videli v prejšnjih poglavjih, nekatere vir (npr. Standard ISO 80000-2) priporočajo, da se za matematične konstante in posebne funkcije uporablja rimska pisava (upravičeno). Seveda, ni smiselno vsakič pisati `\mathrm{\pi}` za konstantno pi ali `\mathrm{e}` za Eulerjevo število. Namesto tega lahko definiramo nove ukaze za te konstante:

Nasvet

Ko definiramo nove ukaze, ki se bodo uporabljali v matematičnem načinu, je priporočljivo, da ne vključimo nobenega simbola, ki označuje matematični način. Na primer, pri definiciji ukaza za konstantno π , je bolje definirati ukaz `\kpi` kot `\mathrm{\pi}` namesto `$_{\mathrm{\pi}}$`.

Če želimo definirati nov ukaz, ki sprejema argumente, moramo določiti vrsto in število argumentov v nizu `<vrsta_argumentov>`. Nekateri najpogostejši tipi argumentov so prikazani v Table 6.

Najlažje tip argumenta je `m`, ki predstavlja obvezni argument. Na primer, lahko definiramo ukaz za barvanje besedila z določenim in barvo. Pri definiciji ukaza, uporabljamo znak `#1` za sklicevanje na prvi argument.

Table 6: Najpogostejši tipi argumentov v `\NewDocumentCommand`

Tip argumenta	Opis
m	Obvezni argument, ki lahko vsebuje več vrstic.
0{<privzeta_vrednost>}	Neobvezni argument, ki lahko vsebuje več vrstic. Če ni podan, ima privzeto vrednost <privzeta_vrednost>.
o	Neobvezni argument. Če ni podan, natisne posebni niz <code>-NoValue-</code> . Običajno v definicij uporaba pogojev za preverjanje, ali je bil argument podan kot je ukaz <code>\IfNoValueTF</code> .
D<levo><desno>{<privzeta_vrednost>}	Neobvezni argument, ki je obdan z <levo> in <desno>. Če ni podan, ima privzeto vrednost <privzeta_vrednost>. Običajno se uporablja znak < in >, za <levo> in <desno>. Deluje kot o, vendar s posebnimi ločevalniki.
d<levo><desno>	Podobno kot D, vendar brez privzete vrednosti. Deluje kot o, vendar s posebnimi ločevalniki.
s	Preklopni argument (switch). Če je prisoten, je vrednost <code>\BooleanTrue</code> , sicer <code>\BooleanFalse</code> . Uporabno, ko definiramo ukaze ‘z zvezdico’ ali ‘brez zvezdice’.

Seveda, lahko definiramo ukaze z več argumenti. Dovolj je da dodamo več tipov argumentov v niz `<vrsta_argumentov>` in uporabimo `#2`, `#3`, ... za sklicevanje na druge argumente.

Če želimo definirati ukaz z neobveznimi argumenti, lahko uporabimo tip `0{<privzeta_vrednost>}`. Ne pozabite, da neobvezni argumenti damo znotraj oklepaj `[]` pri klicu ukaza.

Lahko seveda kombiniramo različne tipe argumentov. Na primer, lahko definiramo ukaz z enim obv

```
::::{grid} 2 2 2 2
```

```
:::{card} LaTeX
```

```
“‘latex
V preambuli:
\NewDocumentCommand{\Zap}{0{n}m}
{#2_{1}, \dots, #2_{#1}}
% V telesu dokumenta:
Nekatere zaporedje imajo veliko elementov:
$\Zap[1000]{a}$,
za večino pa sploh ne vemo,
koliko jih je:
$\Zap{b}$.
```

```
“‘
```

```
:::
```

```
:::{card} PDF
```

```
“‘{figure} ./img/05_eg -ArgsMO -1.png
:label: fig:eg -ArgsMO -1
```

‘ ‘ ‘

: : :

Upoštevajte, da je vrstni red argumentov pomemben. Če je ukaz `\ukaz` definiran z vrstnim redom argumentov `0{m}`, ga moramo klicati z obveznim argumentom najprej, nato pa neobveznim argumentom (če ga želimo podati), to pomeni `\ukaz[<neobvezni>]{<obvezni>}` je pravilen klic, medtem ko `\ukaz{<obvezni>}[<neobvezni>]` ni pravilen klic. Dobra praksa je, da najprej definiramo vse neobvezne argumente, nato pa obvezne argumente.

Včasih želimo da ukaz deluje drugače, če je neobvezni argument podan ali ne. V tem primeru lahko uporabimo tip argumenta `o` in ukaz `\IfValueTF` za preverjanje, ali je bil argument podan.

Namesto ukaza `\IfValueTF`, lahko uporabimo tudi ukaz `\IfValueT`, če želimo izvesti dejanje samo, če je bil argument podan, ali ukaz `\IfValueF`, če želimo izvesti dejanje samo, če argument ni bil podan. Lahko tudi uporabimo ukaze `\IfNoValueTF`, `\IfNoValueT`, in `\IfNoValueF`, ki delujejo nasprotno.

Ko želimo da ukaz sprejme neobvezni argument z določenimi ločevalniki, lahko uporabimo tip argumenta `D<levo><desno>{<privzeta_vrednost>}`. Običajno se uporabljata znaka `<` in `>` kot ločevalnika.

Če namesto `D<levo><desno>{<privzeta_vrednost>}` uporabimo `d<levo><desno>`, dobimo podoben učinek, vendar brez privzete vrednosti (točno tako, kot ko uporabljamo `o`). V tem primeru moramo vedno preveriti, ali je bil argument podan z uporabo ukaza `\IfValueTF` in podobnih.

Nazadnje, lahko tudi uporabljamo znak `s` za niz `<vrsta_argumentov>`, da definiramo preklopni argument oz. različica ukaz *z zvezdico*. Pri definiciji ukaza, lahko uporabimo ukaz `\IfBooleanTF`, da preverimo, ali je bil preklopni argument podan.

Lahko tudi definiramo nova okolja z uporabo ukaza `\NewDocumentEnvironment`, ki ima podobno sintakso kot `\NewDocumentCommand`.

```
\NewDocumentEnvironment{<ime_okolja>}{<vrsta_argumentov>}
{<začetek_definicije>}
{<konec_definicije>}
```

- `<ime_okolja>`: Ime novega okolja, ki ga želimo definirati.
- `<vrsta_argumentov>`: Niz, ki določa vrste argumentov, ki jih okolje sprejema. To deluje skoraj enako kot pri ukazu `\NewDocumentCommand`.
- `<začetek_definicije>`: Koda za začetek okolja, kjer lahko uporabimo argumente znotraj okolja.
- `<konec_definicije>`: Koda za konec okolja.

Naslednji primeri naj bi pojasnili uporabo tega ukaza.

Example 0.1.48.

Lahko tudi definiramo okolja z argumenti. Na primer, lahko definiramo okolje `kraljIme`, ki sprejme en obvezni argument za ime kralja.

Example 0.1.49.

Tukaj imamo problem, ker želimo imeti različne naslove in zaključke za kralja in kraljico. To lahko rešimo z uporabo preklopnega argumenta `s`. Moramo v definiciji okolja uporabiti ukaz `\IfBooleanTF`, da preverimo, ali je bil preklopni argument podan; to lahko storimo v obeh delih definicije okolja.

Pazite, za razliko od nekaterih drugih LaTeX okoljih (npr. `align`), ko uporabljamo okolje z zvezdico, zvezdica gre zunaj oklepajev imenja okolja, t.j. `\begin{kralij}*{Ime}` namesto `\begin{kralij*}{Ime}`. Zvezdica ni potrebna na koncu okolja, t.j. `\end{kralij}` je pravilen klic tudi za okolje z zvezdico.

Example 0.1.50.

Vaje

- Exercise ??: Vaje o ukazih `\input{}` in `\include{}`.
- Exercise ??: Vaja o mešanju barv.
- Exercise ??: Vaje o definiciji novih ukazov z `\NewDocumentCommand`.

0.1.6 Beamer: Predstavitve v LaTeXu

Beamer je LaTeX razširitev za izdelavo drsnic ("slajdov"). Za izdelavo predstavitve z Beamerjem uporabimo razred `beamer` namesto `article`. Nato uporabimo okolja `frame` za vsak posamezen diapozitiv.

Glej tudi: Beamer uporabniški vodič.

V tem poglavju bomo predstavili osnovne koncepte in ukaze za ustvarjanje predstavitev z Beamerjem. Za podrobnejše informacije in napredne funkcije si oglejte uradno Beamer user guide.

Opozorilo

Razred `beamer` uporaba paket `babel` za jezike malo drugače kot običajni razredi dokumentov (kot so `article` ali `report`). V tem razredu je priporočljivo, da je jezik dokumenta določen kot možnost razreda dokumenta (npr. `\documentclass[slovene]{beamer}`) in da se paket `babel` naloži brez možnosti jezika (npr. `\usepackage{babel}`).

Ukaz `\frametitle{}` določi naslov drsnice. Lahko tudi uporabimo ukaz `framesubtitle{}` za podnaslov drsnice. Krajša možnost je, da naslov oz. podnaslov drsnice podamo kot možnost okolja `frame`; Example 0.1.51.

Example 0.1.51.

Pomembne drsnice

Podobno kot pri člankih lahko tudi pri Beamer predstavitev ustvarimo naslovni diapozitiv z uporabo ukazov `\title{}`, `\author{}` in `\date{}` v preambuli dokumenta.

Zdaj ukaz `\maketitle` v telesu dokumenta ustvari naslovni diapozitiv. Ukaz `\maketitle` je enakomerno kot

```
\begin{frame}
\titlepage
\end{frame}
```

Običajno ukaz `\date` nastavimo na ime konference ali dogodek, kjer bo predstavitev prikazana. V razredu `beamer` lahko tudi uporabljamo ukaze `\institute{}`, `\subtitle{}` in `\titlegraphic{}` za dodajanje dodatnih informacij na naslovno drsnico; pogledjte Example 0.1.52.

Example 0.1.52.

Nekatere teme Beamerja (glej Teme in barvne sheme) dodajo informacije o avtorju in datumu na vsako drsnico. Če je vsebina teh polj predolga, ne bo dovolj prostora; v tem primeru lahko dodamo neobvezni kratek argument v ukaze (npr. `\author[<kratek>]{<dolg>}`), ki se bo prikazal v glavi ali nogi drsnice, pogledjte Example 0.1.53.

Example 0.1.53.

Ukaz `\titlegraphic{}` doda sliko na naslovno drsnico. Če želimo dodati logotip na vse drsnice, lahko uporabimo ukaz `\logo{}` v preambuli dokumenta, pogledjte Example 0.1.54. Upoštevajte, da je pozicija logotipa odvisna od uporabljene teme. Poleg tega, nekatere teme ne podpirajo logotipov/grafik na naslovni drsnici.

Example 0.1.54.

V razredu `beamer` lahko tudi uporabimo ukaze za *logične strukture* kot so `\section{}` in `\subsection{}` za organizacijo predstavitve. Ti ukazi ne ustvarijo drsnic, vendar jih lahko uporabimo za generiranje kazala vsebine. To lahko storimo z uporabo ukaza `\tableofcontents` v okolju `frame`, pogledjte Example 0.1.55. Upoštevajte, da razdelek *Literatura* ni v kazalo vključen, ker je definiran z ukazom `\section*{}`.

Example 0.1.55.

Če želimo kazalo prikazati postopoma, lahko k ukazom `\tableofcontents` dodamo možnost `pausesections`. Poglejte Example 0.1.56.

Example 0.1.56.

Ukaze kot `\section` ne ustvarijo drsnic, vendar lahko z ukazom `\sectionpage` znotraj okolja `frame` ustvarimo drsnico, ki prikazuje naslov razdelka.

Druga možnost je ponatis kazala na začetku vsakega poglavja, da bralce spomnimo, kje smo. To lahko storimo z uporabo ukazov `\AtBeginSection[]` in `\tableofcontents[currentsection]` pogledajte Example 0.1.57. Če želimo, da se v kazalu prikažejo samo razdelek (in ne podrazdelek), dodamo ukaz `\tableofcontents` opcijo `hidesubsections`.

Example 0.1.57.

Poleg ukazov `\section` in `\subsection`, Beamer podpira tudi ukaz `\part{}` za večje logične enote. Vsak del predstavitev (definiran z `\part{}`) lahko začne z drsnico, ki prikazuje naslov dela z uporabo ukaza `\partpage` v okolju `frame`. Poleg tega, ima vsak del lahko svoje kazalo vsebine; pogledajte Example 0.1.58.

Example 0.1.58.**Specifikacije prekrivanja**

Izhodna datoteka Beamer je običajno v formatu PDF kar je v principu statična datoteka. Beamer pa omogoča ustvarjanje dinamičnih učinkov z uporabo *specifikacij prekrivanja* (ang. *overlay specifications*).

Predstavitve Beamer sestoji iz serie drsnic, vsaka drsnica pa je lahko razdeljena na več slojev oz. stran (ang. *slides*). V izhodni PDF datoteki vsak sloj ustreza eni strani. Specifikacije prekrivanja so ukazi, ki določajo, kateri deli vsebine so prikazani na določenem sloju.

Najlažji način za uporabo specifikacij prekrivanja je z uporabo ukaza `\pause`, ki ustvari prekinitev na trenutni točki v drsnici. Če dodamo ukaz `\pause` v okolje `frame`, prvi sloj drsnice prikaže vsebino do prva uporaba ukaza `\pause`, drugi sloj prikaže vsebino do druge uporabe ukaza `\pause` in tako naprej. Poglejte Example 0.1.59.

Example 0.1.59.

Ukaz `\pause` je enostaven za uporabo, vendar ponuja omejene možnosti prilagajanja. Za bolj natančno kontrolo nad tem, kdaj se posamezni deli vsebine prikažejo, lahko uporabimo *specifikacije prekrivanja* znotraj ukazov kot so `\textbf`, `\textit`, `\textcolor` in druge (poglejte Table ??).

Sintaksa za specifikacije prekrivanja je naslednja:

```
\ukaz<spec_prek>{<besedilo>}
```

Kjer `\ukaz` predstavlja ukaz, ki ga želimo nadzorovati (npr. `\textbf`), `spec_prek` pa določa, na katerih slojih bo ukaz uporabljen. Na primer, `\textbf<2>{<besedilo>}` bo prikazalo `<besedilo>` krepko le na drugem sloju drsnice (ostali sloji bodo prikazali `<besedilo>` v običajnem slogu); pogledajte Example 0.1.60.

Example 0.1.60.

Nekatere ukaze, kot so `\item` v okolju `itemize` in `enumerate`, imajo vgrajeno podporo za specifikacije prekrivanja; pogledajte Example 0.1.61.

Example 0.1.61.

Ukaz `\includegraphics` podpira tudi specifikacije prekrivanja, kar omogoča prikazovanje različnih slik na različnih slojih drsnice. To je uporabno za simulacijo animacij z zaporednim prikazovanjem slik; pogledajte Example 0.1.62.

Example 0.1.62.

`stick.zip` vsebuje slike, uporabljene v zgornjem primeru.

Naslednje tri ukaze delujejo zelo podobno. Prikažejo `<besedilo>` v skladu z `<spec_prek>`. Razlika med njimi je v tem, kako se obnašajo, ko besedilo ni prikazano.

- `\only<spec_prek>{<besedilo>}`: Prikaže `<besedilo>` samo na slojih, določenih z `<spec_prek>`. Na drugih slojih `<besedilo>` ni prisotno (ne zavzame prostora).
- `\visible<spec_prek>{<besedilo>}`: Prikaže `<besedilo>` na slojih, določenih z `<spec_prek>`, in na drugih slojih ohrani prostor, ki ga `<besedilo>` zavzema (vendar je **nevidno**).
- `\uncover<spec_prek>{<besedilo>}`: Prikaže `<besedilo>` na slojih, določenih z `<spec_prek>`, in na drugih slojih ohrani prostor, ki ga `<besedilo>` zavzema (vendar je **prozoren**).

Razlika med `\visible` in `\uncover` je odvisno od uporabe *prosojnosti* (ang. *transparency*). To se nadzoruje z ukazom `\setbeamercovered{<možnost>}` v preambuli dokumenta. Možnosti vključujejo:

- `invisible`: Besedilo je popolnoma nevidno (privzeta možnost).
- `transparent=<neprosojnost>`: Besedilo je prikazano z določeno stopnjo prosojnosti: `<neprosojnost>` je vrednost med 0 (popolnoma prozorno) in 100 (popolnoma neprozorno); privzeta vrednost je 15.
- `dynamic`: Vse prekrita besede postanejo precej prosojne, vendar na dinamičen način. Daljši čas potreben za odkritje besedila, močnejša je prosojnost. Poglejte Example 0.1.63.

Example 0.1.63.

Obstajajo še drugi ukazi za specifikacije prekrivanja, kot so:

- `\invisible<spec_prek>{<besedilo>}`: Točno nasprotje ukaza `\visible`. Prikaže `<besedilo>` na slojih, določenih z `<spec_prek>`, in na drugih slojih besedilo ni prisotno.
- `\alt<spec_prek>{<besedilo1>}{<besedilo2>}`: Prikaže `<besedilo1>` na slojih določenih z `<spec_prek>`, in `<besedilo2>` na ostalih slojih.
- `\temporal<spec_prek>{<besedilo1>}{<besedilo2>}{<besedilo3>}`: Prikaže `<besedilo1>` pred prvi sloji določenimi z `<spec_prek>`, `<besedilo2>` na slojih določenih z `<spec_prek>`, in `<besedilo3>` po zadnji sloji določeni z `<spec_prek>`.

V Example 0.1.64 so prikazani ti ukazi v akciji.

Example 0.1.64.

Nekatere okolje v Beamerju podpirajo specifikacije prekrivanja. Sintaksa je običajno `\begin{okolja}<spec_prek>` in rezultat je, da se celotno okolje prikaže samo na slojih določenih z `<spec_prek>`.

Vsak od ukazov `\only`, `\alt`, `\visible`, `\uncover` in `\invisible` ima svojo različico okolja, in sicer, `onlyenv`, `altenv`, `visibleenv`, `uncoverenv` in `invisibleenv`. Vsi razen `altenv` ima sintakso

```
\begin{okolja}<spec_prek>
  <vsebina>
\end{okolja}
```

in delujejo podobno kot njihovi ukazni ekvivalenti. Okolje `altenv` ima sintakso

```
\begin{altenv}<spec_prek>
{<begin_privzeto>}{<end_privzeto>}
{<begin_alternativno>}{<end_alternativno>}
  <vsebina>
\end{altenv}
```

V določenih slojih določenih z `<spec_prek>`, se `<vsebina>` obdana med `<begin_privzeto>` in `<end_privzeto>`, medtem ko se v ostalih slojih prikaže vsebina obdana z `<begin_alternativno>` in `<end_alternativno>`.

Lahko uporabimo ukaze kot je `\only` za dinamično spremeniti vsebino drsnice, npr.

```
\only<1>{Besedilo za prvi sloj.}
\only<2>{Besedilo za drugi sloj, ki je lahko daljše. Če je besedilo daljše, se okvir prilago
\only<3>{Tretji sloj ima spet krajše besedilo.}
```

Težava pri tem pristopu je, da lahko pride do nadležnih razlik v višini elementov, kar lahko povzroči, da se celoten okvir »trese« od sloja do sloja.

Za rešitev tega problema lahko uporabimo okolje `overlayarea`, ki ima naslednjo sintakso:

```
\begin{overlayarea}{<širina>}{<višina>}
  <vsebina>
\end{overlayarea}
```

To okolje ustvari območje `<širina>` krat `<višina>`, v katerem se vsebina lahko dinamično spreminja brez vpliva na višino okvirja drsnice; pogledjte Example 0.1.65.

Example 0.1.65.

Vsebina predstavitev

Seznami Navadna okolja za sezname, kot so `itemize`, `enumerate` in `description`, so podprta v Beamerju.

Seveda lahko uporabimo tudi ukaz `\pause` ali specifikacije prekrivanja kot prikazano v Example 0.1.61 za postopno prikazovanje elementov seznama. Druga možnost je uporaba specifikacije `[<+>]` v okolju seznama, kar samodejno prikaže vsak element na naslednjem sloju; pogledjte Example 0.1.66.

V tem primeru tudi pokažemo uporabo ukaza `\vspace{<višina>}` za ustvarjanje navpičnega prostora med seznami in ukaza `\vfill`, ki potisne vsebino navzdol do dna okvirja drsnice.

Example 0.1.66.

Če je seznam predolg, da bi ga lahko prikazali na eni drsnici. V tem primeru lahko uporabimo pristop, ki ga prikazuje Example 0.1.67.

Example 0.1.67.

Stolpci Beamer omogoča ustvarjanje stolpcev z uporabo okolja `columns`. Vsak stolpec je definiran z okoljem `column`, ki ima naslednjo sintakso:

```
\begin{columns}
  \begin{column}<spec_prek>{<širina>}
    <vsebina stolpca 1>
  \end{column}
  \begin{column}<spec_prek>{<širina>}
    <vsebina stolpca 2>
  \end{column}
  ...
\end{columns}
```

Kjer `<širina>` določa širino stolpca (lahko je absolutna vrednost, npr. `5cm`, ali relativna vrednost, npr. `0.5\textwidth`), `<spec_prek>` pa so opcijske specifikacije prekrivanja, ki določajo, na katerih slojih bo stolpec prikazan. Pogledjte Example 0.1.68.

Example 0.1.68.

Ta pristop lahko uporabimo za ustvarjanje drsnic z besedilom na eni strani in sliko na drugi strani; pogledjte Example 0.1.69.

Example 0.1.69.

Tabele Na splošno večina ukazov, ki smo jih opisali v poglavju [Osnovna okolja](#) za ustvarjanje tabel v LaTeX-u deluje tudi v Beamerju, dokler naložimo ustrezne pakete (npr. `booktabs`, `array`, `multrow`).

Poleg tega lahko uporabimo specifikacije prekrivanja za postopno prikazovanje vrstic ali stolpcev v tabeli; pogledjte Example 0.1.70.

Example 0.1.70.

Če je tabela preširoka ali predolga za drsnico, lahko uporabimo okolje `resizebox` iz paketa `graphicx`. Sintaksa je `\resizebox{<širina>}{<višina>}{<vsebina>}`, kjer lahko uporabimo `!` za samodejno prilagoditev višine ali širine glede na drugo dimenzijo. Pogledjte Example 0.1.71.

Example 0.1.71.

Bloki Beamer ponuja okolje `block` za ustvarjanje poudarjenih vsebinskih blokov znotraj drsnic. Sintaksa je naslednja:

```
\begin{block}{<naslov bloka>}
  <vsebina bloka>
\end{block}
```

Poleg tega Beamer ponuja posebne vrste blokov, kot so `exampleblock` in `alertblock`, ki imajo različne sloge za poudarjanje vsebine. Slog bloka je odvisen od izbrane teme Beamerja. Pogledjte Example 0.1.72.

Example 0.1.72.

Okolje za teoreme, kot so `theorem`, `lemma`, `definition`, `proof`, itd. so prav tako podprta v Beamerju in imajo slog, ki je podobno kot bloki. Pogledjte Example 0.1.73.

Example 0.1.73.

Lahko tudi definiramo svoje matematične bloke z uporabo ukaza `\newtheorem` točno tako, kot v običajnem LaTeXu.

Teme in barvne sheme

Beamer ponuja različne teme in barvne sheme za prilagoditev videza predstavitve. Teme določajo splošno postavitev in slog drsnic, in barvno paletu.

Obstajajo štiri parametri za prilagoditev videza predstavitve v Beamerju:

- Tema barv (ang. `color theme`): Določa barvno shemo predstavitve.
- Tema pisave (ang. `font theme`): Določa slog pisave.
- Notranje teme (ang. `inner theme`): Določa slog notranjih elementov, kot so sezname, bloki, kazalo, itd.
- Zunanje teme (ang. `outer theme`): Določa slog zunanjih elementov, kot so glave, noge, okvirji drsnic, itd.

Poleg teh štirih parametrov lahko izberemo tudi celotno temo, ki združuje vse te vidike v eno samo temo. Večinoma časa, je dovolj, da izberemo samo temo, saj bo ta samodejno nastavila ustrezne barvne sheme, pisave in notranje ter zunanje teme in če želimo, lahko dodatno prilagodimo posamezne parametre.

Teme se naložijo z ukazom `\usetheme{<ime_teme>}`, medtem ko se posamezni parametri naložijo z ukazi

- `\usecolortheme{<ime_barvne_sheme>}`
- `\usefonttheme{<ime_pisave>}`
- `\useinnertheme{<ime_notranje_teme>}`
- `\useoutertheme{<ime_zunanje_teme>}`

Število možnosti za te ukaze je ogromno in nemogoče je prikazati vse relevantne primere. Lahko pa ponudimo zunanje vire za raziskovanje različnih tem.

- Beamer Theme Matrix: Interaktivno orodje za raziskovanje različnih kombinacij tem in barvnih shem v Beamerju.
- Beamer Theme Gallery: Galerija različnih tem Beamer, tem pisave in tem barv.
- Beamer User Guide: Del III vodiča za uporabnike Beamerja vsebuje podrobne informacije o temah in barvnih shemah.
- Overleaf beamer templates: Sodobne teme Beamer na Overleafu.

Vaje

0.2 GeoGebra

Table 7: Primerjava različnih aplikacij Geogebra

aplikacije / funkcije	Scientific	Graphing	Geometry	3D	CAS	Suit	Classic
Numerični izračuni	x	x	x	x	x	x	x
Funkcije	x	x	x	x	x	x	x
Operacije z ulomki	x	x	x	x	x	x	x
Grafično prikazovanje		x	x	x	x	x	x
Drsniki		x	x	x	x	x	x
Vektorji in matrike		x	x	x	x	x	x
Tabele		x			x	x	x
Geometrijske konstrukcije			x	x	x	x	x
3D prikazovanje				x		x	x
Verjetnost in statistika						x	x
Odvodi in integrali				x	x	x	x
Reševanje enačb				x	x	x	x
Simbolni izračuni				x	x	x	x
Spreadsheets							x

0.2.1 GeoGebra: Uvod

Kaj je Geogebra?

Geogebra je brezplačno dinamično matematično orodje, ki omogoča raziskovanje in vizualizacijo različnih matematičnih konceptov.

Trenuto, Geogebra je popoln paket za učenje in poučevanje matematike. Ponuja različne aplikacije, ki pokrivajo področja, kot so geometrija, algebra, statistika in kalkulus. Vse te aplikacije je mogoče namestiti lokalno ali dostopati do njih prek spleta:

- Scientific Calculator: Za izračune z ulomki, statistiko in osnovnimi matematičnimi funkcijami.
- Graphing Calculator: Za vizualizacijo enačb in funkcij z interaktivnimi grafikoni.
- Geometry: Za dinamične geometrijske koncepte in konstrukcije v ravnini.
- 3D Calculator: Za raziskovanje tridimenzionalnih oblik in površin.
- CAS Calculator: Za simbolne izračune in reševanje enačb.
- Calculator suit: Kombinacija različnih orodij za širok spekter matematičnih nalog. Idealna za učence in študente.
- Geogebra Classic: Vse-v-enem aplikacija, ki združuje vse zgoraj omenjene funkcije. Idealna za avtorije učnih gradiv.

Različne možnosti, ki jih ponuja vsako od orodij, so navedene v Table 7.

Mi bomo v nadaljevanju uporabljali predvsem Geogebra Classic, saj omogoča uporabo vseh orodij na enem mestu.

Kako začnemo z Geogebro?

Geogebra lahko uporabimo na dva načina: prek spletnega brskalnika ali z namestitvijo aplikacije na računalnik (ali telefon/tablet).

V vsakem primeru bomo uporabili račun Geogebra, ki nam omogoča shranjevanje naših delovnih zvezkov v oblak in dostop do njih od kjerkoli.

Zdaj imate dve možnosti za začetek uporabe Geogebre:

1. Uporaba spletnega brskalnika:

- Obiščite Geogebra Classic.
- Kliknite na gumb 
Menu v zgornjem levem kotu.
- Prijavite se s svojim računom z klikom na 
Sign in.

2. Namestitev aplikacije:

- Prenesite aplikacijo Geogebra Classic za vaš operacijski sistem z uradne strani za prenos.
- Namestite aplikacijo in jo odprite.
- Prijavite se s svojim računom z klikom na gumb 
Menu v zgornjem levem kotu in nato na 
Sign in.

Ko odprete Geogebro in se prijavite, boste videli glavno delovno okolje. Privzeto je prikazi 
Graf (ang. 

Graphing perspective), ki omogoča risanje funkcij in geometrijskih objektov. Lahko jo vidite tudi spodaj:

Opomba.

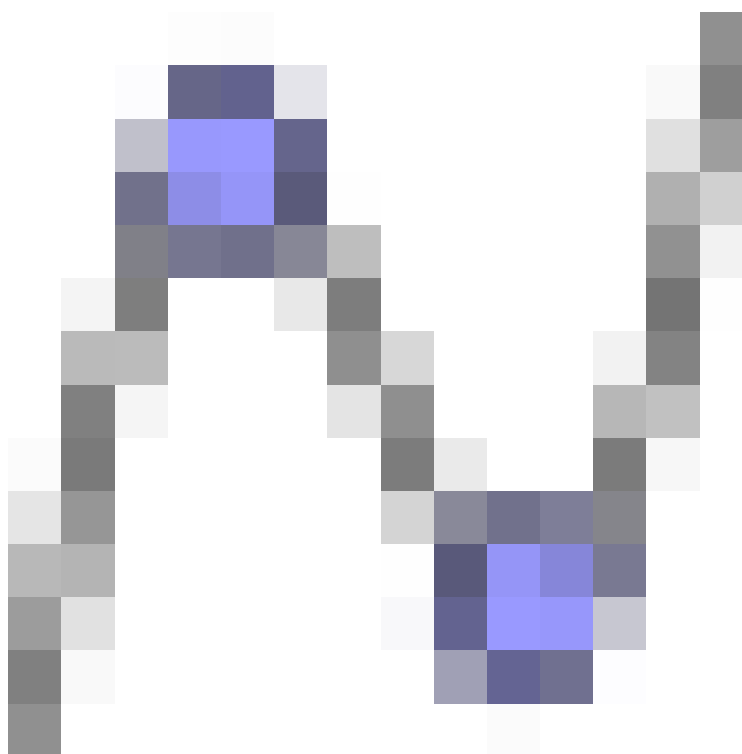


Graphing perspective je eden izmed več prikazi, ki jih ponuja Geogebra. Vsaka aplikacija Geogebre je sestavljena iz različnih prikazov, tako da lahko ustvarimo gradiva za različne aplikacije. Različne prikaze lahko izberemo s klikom na gumb 
Menu v zgornjem levem kotu in nato izbiro možnosti 
Perspectives.

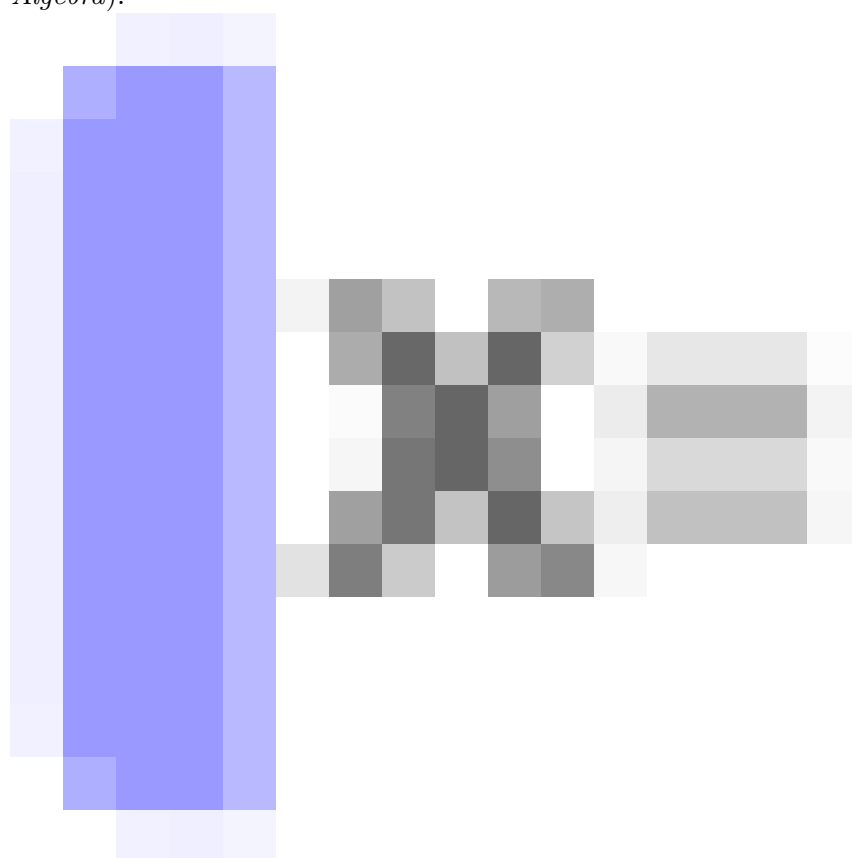
Vsak prikaz ponuja različne *pogled*e (ang. *views*). Lahko jih dodamo ali odstranimo glede na naše potrebe s klikom na gumb 

Menu v zgornjem levem kotu in nato izbiro možnosti 
View. Različni pogledi vključujejo:

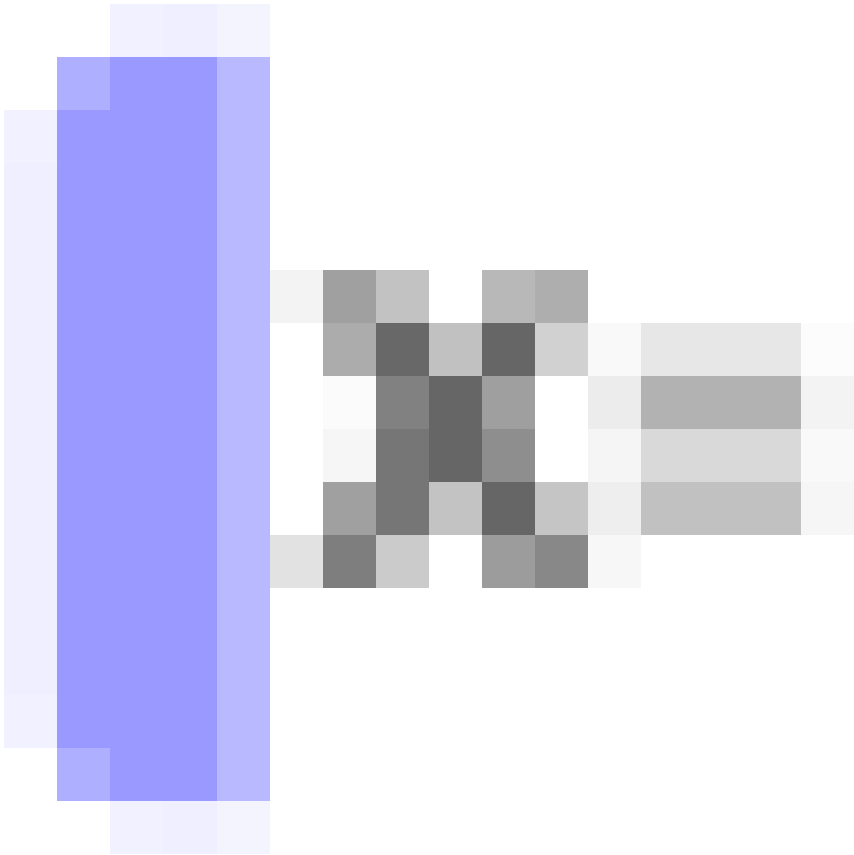
- 



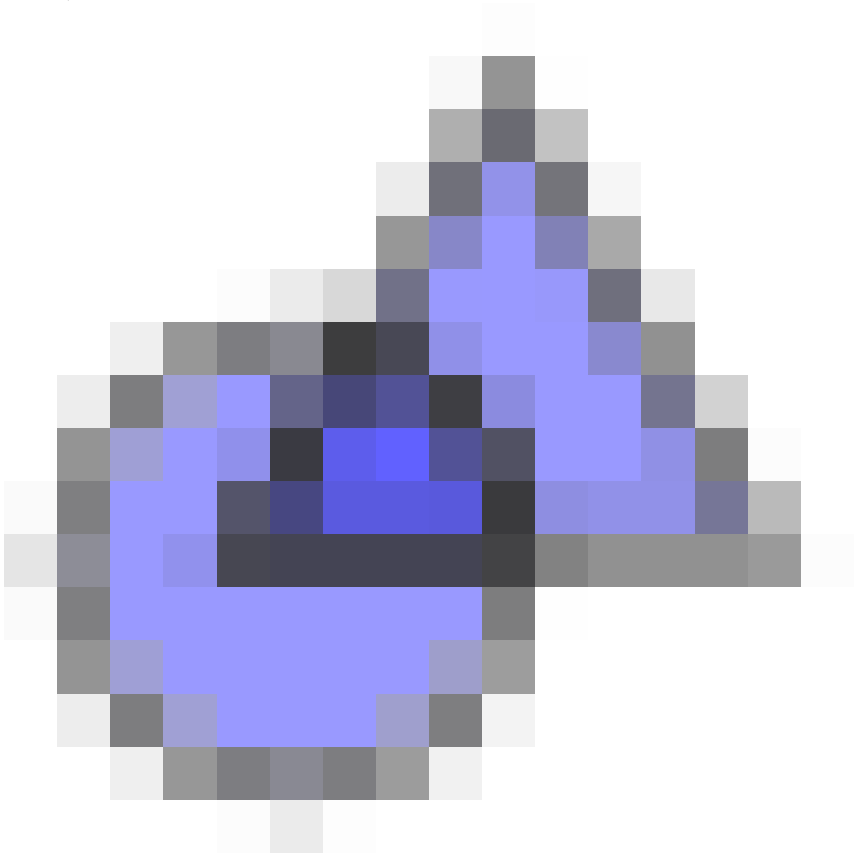
*Algebrsko okno (ang.
Algebra).*



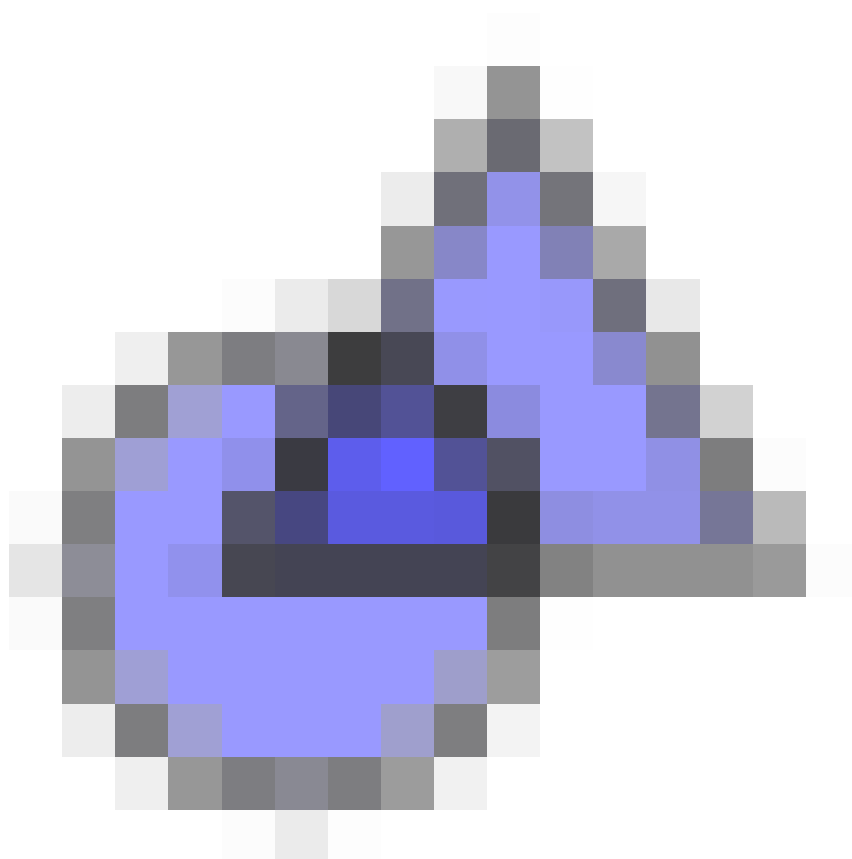
•



Simbolno računanje (ang. CAS).



•

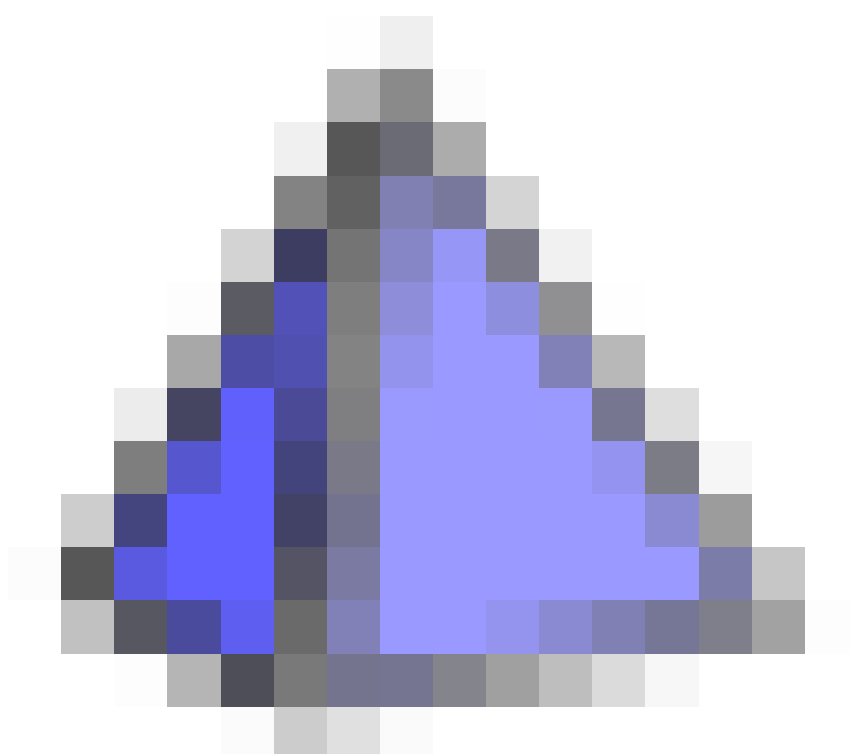


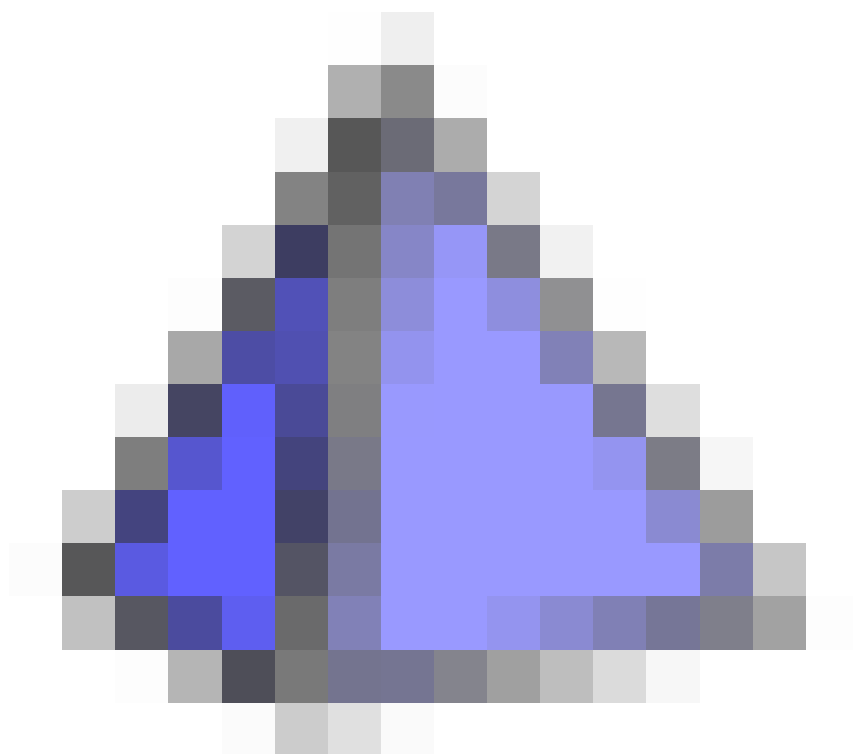
*Risalna površina (ang.
Graphics).*



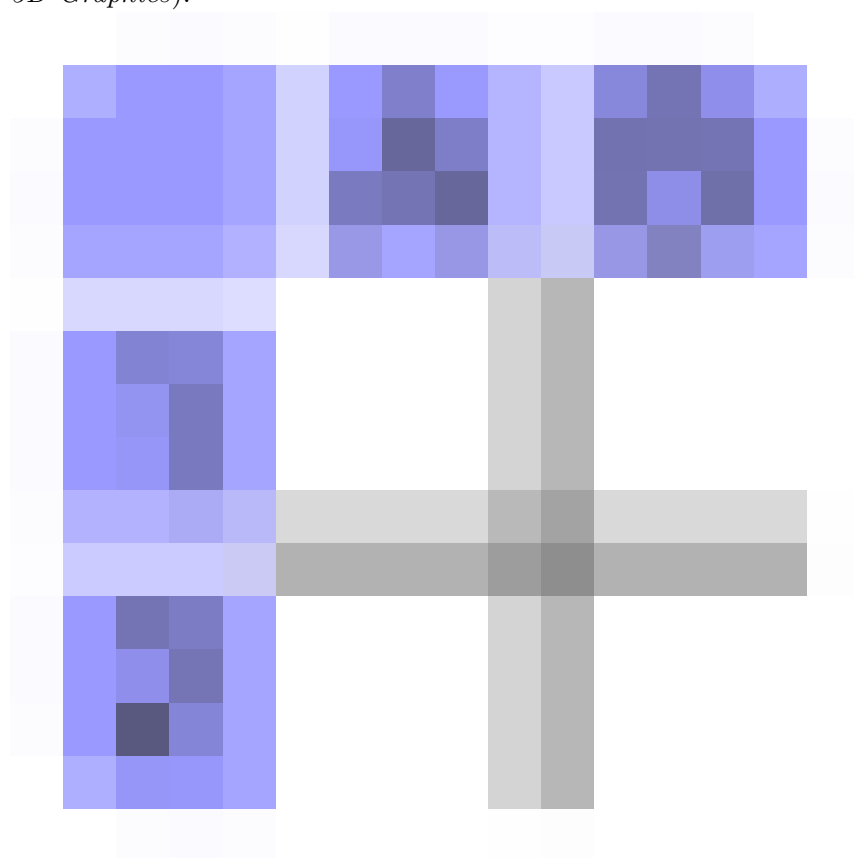


*Grafika 2 (ang.
Graphics 2).*

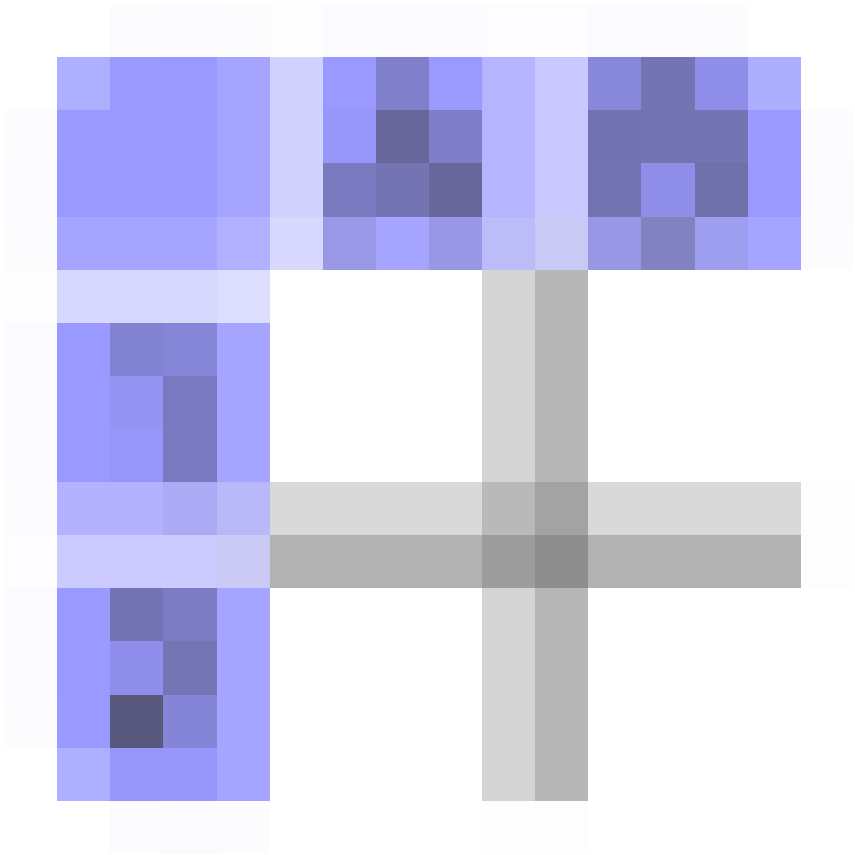




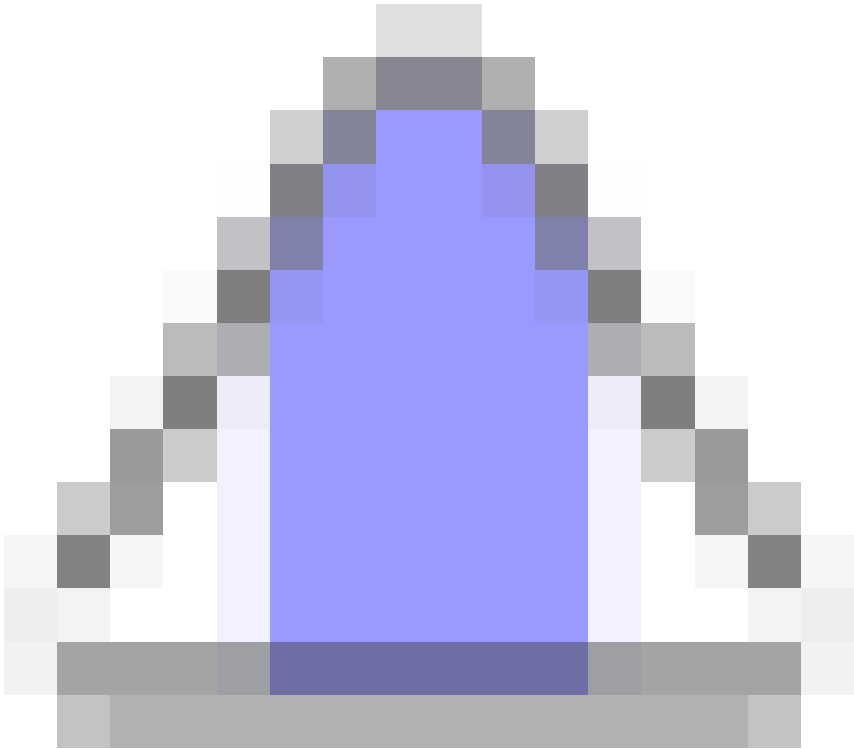
*3d grafika (ang.
3D Graphics).*



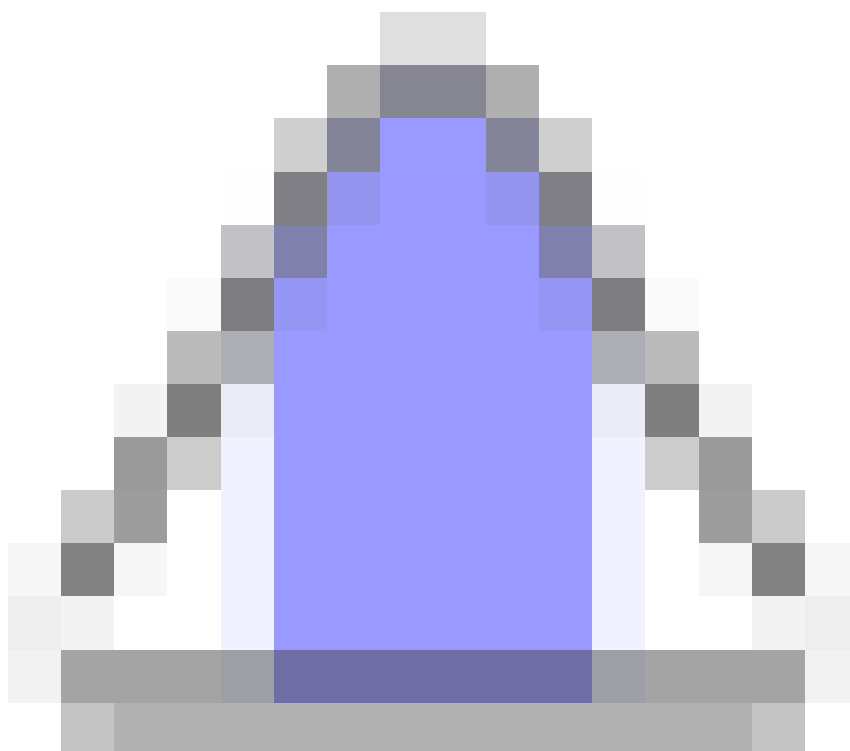
•



*Preglednica (ang.
Spreadsheet).*



•



Računanje verjetnosti (ang.
Probability Calculator).

Drugi vnosi nad gumbom ☰

Menu omogočajo dostop do različnih orodij in nastavitev Geogebre, kot so:

-  *Datoteka* (ang.  *File*): Ustvarjanje, odpiranje, shranjevanje in izvoz datotek.
-  *Uredi* (ang.  *Edit*): Razveljavi, ponovi, kopiraj, prilepi in druge osnovne funkcije urejanja.
-  *Nastavitve* (ang.  *Settings*): Prilagajanje nastavitev Geogebre, kot so jezika, velikosti pisave in drugih možnosti.

Orodarne (ang. *Toolbar*):



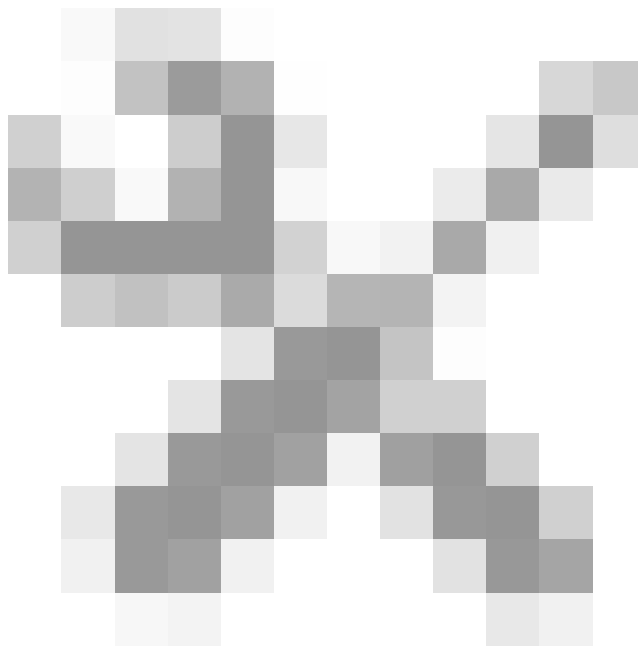
Slog vrstica (ang. *Stylebar*):



Razveljavi (ang. *Undo*):



Ponovno naredi (ang. *Redo*):



- **Orodja** (ang. 
Tools): Dostop do različnih orodij za risanje in konstrukcije.
- **Pomoč** (ang. 
Help and Feedback): Dostop do dokumentacije, vadnic in podpore skupnosti Geogebra.
- **Račun** (ang. 
Account): Prijava in odjava iz računa Geogebra.

Opomba

Geogebra se lahko uporablja v slovenskem jeziku, vmesnik je preveden, prav tako pa tudi ukazi, žal pa ni dokumentacije v slovenskem jeziku za ukaze (ki jih uporabljamo za ustvarjanje gradiva). Zaradi tega bomo vmesnik ohranili v angleščini, vendar upoštevajte, da če želite Geogebro uporabljati z otroki ali učenci, lahko jezik spremenite v meniju  *Settings*.

Poleg gumba 

Menu, pomembni del orodjarne so tudi:

- Točke so definirane z velikimi črkami (npr. A , B , C), medtem ko so premice, daljice in krožnice definirane z malimi črkami (npr. a , b , c).
- Pri vnosu koordinat točk uporabite oklepaje in vejice, npr. $(2,3)$ za točko z x koordinato 2 in y koordinato 3.
- Ukaz $v = (1,2)$ ne definira točke, ampak vektor z komponentama 1 in 2.
- Uporabite za samodejno dokončanje imen ukazov in objektov.
- Uporabite in za navigacijo po zgodovini ukazov v vrstici za vnos.
- Z gumbom  *Help* (sl. *Pomoč*) (na desno od vrstice za vnos) lahko dostopate do dokumentacije Geogebre za ukaze.

V Exercise ?? ste se naučili

- Ustvariti geometrijske objekte z uporabo ukazov v vrstici za vnos.
- Meriti kotov in dolžin in z uporabo orodje.
- Prilagoditi besedilo z uporabo LaTeX formul.
- Nastavitev **Starting point** za pozicioniranje besedila.

Geogebra nam omogoča vizualizacijo osnovnih geometrijskih konstrukcij in iskanje vzorcev v njih. Na primer, pri konstrukciji v Exercise ?? lahko opazimo, da je trikotnik ABC vedno enakostraničen, ne samo za izbiro točko $A = (0,0)$ in $C = (4,0)$, ampak za katerokoli izbiro točk A in C . To lahko vidimo, če povlečemo točki A in C z orodjem .

Move. Opazimo da so dolžini stranici AB , BC in CA vedno enaki. V Geogebri je to znano kot **test vlečenja** (ang. *dragging test*), ki nam pomaga pri odkrivanju in dokazovanju geometrijskih lastnosti.

V prešnji konstrukciji smo uporabili **test vlečenja** (ang. *dragging test*), ki nam pomaga pri odkrivanju in dokazovanju, da je trikotnik ABC vedno enakostraničen. S testom vlečenja lahko tudi vidimo, da so vsi koti v enakostraničnem trikotniku enaki in merijo 60° . To sledi iz dejstva, da je vsota notranjih kotov v trikotniku vedno 180° in da so vsi trikotniki enakostranični.

Algebrsko okno, proste točke vs. odvisne točke in kontrolni okvirček

Vrstica za vnos ni edini način za interakcijo z Geogebro z uporabo ukazov. Lahko uporabimo tudi  *Algebrsko okno* (ang. *Algebra View*), ki prikazuje seznam vseh objektov v konstrukciji skupaj z njihovimi lastnostmi in vrednostmi.

Upoštevajte, da so točke A in B **proste točke** (ang. *free points*), ker jih lahko premikamo kjerkoli v grafičnem oknu. Vse ostale točke v konstrukciji so **odvisne točke** (ang. *dependent points*), ker so določene z drugimi objekti v konstrukciji in se premikajo glede na spremembe teh objektov. Proste točke so običajno modre barve, medtem ko so odvisne točke sive barve. Seveda, to lahko spremenimo z uporabo vrstice za zamenjavo sloga ali v nastavitvah objekta, in torej barva ni zanesljiv indikator vrste točke. V algebrskem oknu lahko vidimo katere objekte so proste in katere odvisne, saj so uporabimo gump *Sort by* in torej *Dependency* v vrstico za zamenjavo sloga algebrskega okna.

Kot smo videli, lahko uporabimo orodje .

Show/Hide Object za prikaz ali skrivanje objektov v grafičnem oknu. To lahko storimo tudi v algebrskem oknu, kjer lahko kliknemo na krog levo od imena objekta za prikaz ali skrivanje objekta. Poleg tega, lahko uporabimo tudi kontrolni okvirček (ang. *checkbox*), ki ga lahko ustvarimo z orodjem .

Check Box ali z ukazom `CheckBox()`. Kontrolni okvirček vstavirte *Check Box* v grafično okno in booleano vrednost v algebrskem oknu. Nato lahko uporabimo to vrednost za nadzor vidljivosti drugih objektov v konstrukciji.

Izvoz konstrukcij kot slike.

Lahko izvozimo naše konstrukcije iz Geogebre kot slike v različnih formatih, kot so PNG, SVG ali PDF. Za izvoz konstrukcije kot slike, sledite tem korakom:

1. Pojdite na meni 

File in izberite 

Download as...

2. V pogovornem oknu izberite želeni format slike (npr. `.png`, `.svg` ali `.pdf`).
3. Posebni format je `PGF/TikZ`, ki omogoča izvoz konstrukcije kot kode, ki jo lahko uporabimo v LaTeX dokumentih.

Vaje

- Exercise ??: Ustvarjanje računa Geogebra.
- Exercise ??: Vaje za spoznavanje orodij Geogebre.
- Exercise ??: Vaje za uporabo vrstice za zamenjavo sloga.
- Exercise ??: Vaje za spoznavanje ukazov Geogebre.
- Exercise ??: Vaje za dokazovanje lastnosti trikotnika z uporabo Geogebre.
- Exercise ??: Vaje za dokazovanje vsote notranjih kotov v trikotniku z uporabo Geogebre.
- Exercise ??: Vaje za ustvarjanje petkotnika in zvezde z uporabo Geogebre.
- Exercise ??: Vaje za uporabo kontrolnega okvirčka za nadzor vidljivosti objektov v Geogebri.

0.2.2 Dinamična geometrija z GeoGebro

V poglavju [GeoGebra: Uvod](#) smo spoznali osnove dela z GeoGebro. V tem poglavju bomo raziskali, kako ustvariti animacije in dinamične konstrukcije z uporabo GeoGebre.

Točka na objektu

Najlažje način za ustvarjanje dinamične konstrukcije je uporaba točke, ki je vezana na drug objekt. Točko lahko postavimo na premico, krožnico ali katerikoli drug objekt, kar omogoča, da se točka premika znotraj omejitve tega objekta.

Ko ustvarimo točko na krivulji (kot so premice, krožnice, graf funkcij itd.), lahko to točko animiramo. To naredimo tako, da izberemo točko in kliknemo na gumb .

Play v algebrskem oknu ali uporabimo ukaz `StartAnimation(<točka>)`. S tem se bo točka začela premikati po objektu, na katerem je bila ustvarjena.

Na žalost GeoGebra ne omogoča animacije točk na daljših objektih, kot so mnogokotniki. Vendar pa lahko ustvarimo drsnike, ki nam omogočajo nadzor nad položajem točke na takšnih objektih s pomočjo parametra.

Drsniki

V resnični, drsniki so samo orodja za spreminjanje vrednosti spremenljivk. Vendar pa so zelo uporabni pri ustvarjanju dinamičnih konstrukcij, saj nam omogočajo, da spreminjamo vrednosti in opazujemo, kako se konstrukcija spreminja.

Drsniki imajo tri glavne lastnosti: **Min** (minimalna vrednost), **Max** (maksimalna vrednost) in **Increment** (prirastek). Te lastnosti določajo obseg vrednosti, ki jih drsnik lahko zavzame, in kako hitro se vrednost spreminja, ko premikamo drsnik.

Za ustvarjanje drsnika lahko uporabimo orodje .

Slider (*Drsnik*) in dobimo okno za nastavitev lastnosti drsnika. Drsnike lahko ustvarimo tudi z vnosom imena spremenljivke v vnosno vrstico, na primer $a=5$, kar ustvari spremenljivko z imenom a in začetno vrednostjo 5. Nato lahko kliknemo na tri pike poleg imena spremenljivke v algebrskem oknu in nato *Create slider* (*Ustvari drsnik*). To ustvari drsnik z privzetimi lastnostmi (**Min**=-5, **Max**=5, **Increment**=0.1), ki jih lahko spremenimo z dvojnim klikom na drsnik (v grafičnem pogledu) ali z desnim klikom na ime spremenljivke v algebrskem oknu in izbiro *Settings* (*Nastavitve*). V oknu z nastavitvami lahko spremenimo lastnosti drsnika, kot so **Min**, **Max** in **Increment**, in tudi druge lastnosti, kot so barva, slog in vidnost.

Drsniki lahko tudi uporabimo za ustvarjanje animacij. To naredimo tako, da izberemo drsnik in kliknemo na gumb .

Play v algebrskem oknu ali uporabimo ukaz `StartAnimation(<drsnik>)`. S tem se bo vrednost drsnika začela spreminjati samodejno z določenim prirastkom, kar omogoča ustvarjanje animacij v konstrukciji.

Zaporedja

Ukaz `Sequence()` omogoča ustvarjanje zaporedij objektov v GeoGebri. Splošna oblika ukaza je `Sequence(<izraz>, <spremenljivka>, <začetek>, <konec>, <korak>)`, kjer `<izraz>` je izraz (običajno ukaz GeoGebre), ki ga želimo ponoviti, `<spremenljivka>` je spremenljivka, ki se spreminja v zaporedju, `<začetek>` in `<konec>` določata obseg vrednosti za spremenljivko, in `<korak>` določa, za koliko se spremenljivka poveča v vsakem koraku. Variantne ukaza so:

- `Sequence(<konec>)` - ustvari zaporedje števil od 1 do `<konec>`.
- `Sequence(<začetek>, <konec>)` - ustvari zaporedje števil od `<začetek>` do `<konec>`.
- `Sequence(<izraz>, <spremenljivka>, <začetek>, <konec>)` - ustvari zaporedje z privzetim korakom 1.

Ukaz `Sequence()` ustvari zaporedje objektov, če hočemo izbrati posamezne objekte iz zaporedja, lahko uporabimo ukaz `Element(<zaporedje>, <indeks>)`, kjer `<zaporedje>` je zaporedje objektov in `<indeks>` je indeks objekta, ki ga želimo izbrati (indeksi se začnejo pri 1).

Če hočemo izgraditi več objektov (namesto zaporedje objektov), lahko uporabimo naslednje način:

- Ustvarimo zaporedje objektov z ukazom `Sequence()`, npr. `kroznice = Sequence(Circle((0,0), 1), i, 1, 5)`, ki ustvari zaporedje krožnic s polmeri od 1 do 5.
- Nato uporabimo ukazi `Sequence()` in `Element()` za ustvarjanje zaporedja ukazov, npr. `kroU=Sequence("K_{"+i+"} = Element(kroznice, "+i+")", i, 1, 5)`, ki ustvari zaporedje ukazov za vsako krožnico v zaporedju z imenom `krogU`. V tem primeru ustvarimo ukaze `K_{1} = Element(kroznice, 1)`, `K_{2} = Element(kroznice, 2)`, itd.
- Na koncu uporabimo ukaz `Execute()` za izvedbo vsakega ukaza.

Ta način lahko uporabimo za ustvarjanje zaporedje ukazov, da ni nujno ustvariti objekte. Na primer z ukazom `barv=Sequence("SetDynamicColor(L_{"+i+"}, random(), random(), random(), 0.5)", i, 1, 40)`, ustvarimo zaporedje ukazov za nastavitev naključne barve za vsako daljico, nato pa z ukazom `Execute(barv)` izvedemo te ukaze in tako nastavimo barve za vse daljice naenkrat. *Pozkusite to sami!*

Zaporedje in drsniki postanejo zelo močna orodja, ko jih kombiniramo skupaj. Na primer, lahko ustvarimo drsnik, ki določa število objektov v zaporedju, in nato uporabimo ta drsnik za ustvarjanje dinamične konstrukcije, ki se spreminja glede na vrednost drsnika. Ta način bomo raziskali v naslednjih vajah.

Vaje

- Exercise ?? - Ustvarite dinamični paralelogram in opazujte, kako se spreminja njegova površina.
- Exercise ?? - Ustvarite premico z drsniki za naklon in presečišče z osjo y ter opazujte, kako se spreminja premica.
- Exercise ?? - Ustvarite dinamično konstrukcijo, ki prikazuje veljavnost trikotne neenakosti s pomočjo drsnikov in dinamičnih barv.
- Exercise ?? - Ustvarite animacijo z uporabo drsnikov za premikanje točk v ravnini.
- Exercise ?? - Ustvarite dinamično konstrukcijo za seštevanje dveh celih števil z uporabo drsnikov.
- Exercise ?? - Ustvarite zaporedje točk na krožnici z uporabo ukaza `Sequence()`.
- Exercise ?? - Uporabite ukaz `Sequence()` za ustvarjanje vzorca premic v GeoGebri.
- Exercise ?? - Uporabite ukazi `Element()` in `Execute()` za ustvarjanje posamezni objektov iz zaporedje.

0.2.3 Analiza z GeoGebro

Sledovi (traces in loci)

Sledovi (ang. *traces*) in loci (ang. *loci*) so uporabna orodja v GeoGebri za analizo geometrijskih in funkcijskih lastnosti. Sledovi omogočajo sledenje poti točke, ki se premika glede na druge objekte, medtem ko loci prikazujejo množico točk, ki izpolnjujejo določene pogoje. Razlika med sledovi in loci je v tem, da sledovi prikazujejo pot posamezne točke, medtem ko loci prikazujejo celotno množico točk, ki izpolnjujejo določene pogoje. Najlažje je, da to ponazorimo z naslednjo konstrukcijo:

Glej tudi

Peucellier-Lipkinov mehanizem je opisan tudi na Wikipediji (ang.). To je mehanizem, ki pretvarja krožno gibanje v ravno linijsko gibanje. Uporablja se lahko za risanje ravnih črt brez uporabe ravnila, poleg tega, lahko tudi uporabimo za risanje krožnic brez uporabe kompasa.

Lahko tudi uporabimo orodja 

Image (Slika) za uvoz slike mačke in nato sledimo poti določene točke na sliki. To je prikazano v naslednjem konstrukciji:

Če želite, lahko to konstrukcijo odprete v GeoGebri tukaj. **Poskusite sami!**, Če želite, lahko uporabite moje slike: wall.png in cat.png.

Funkcije

GeoGebra ima veliko vnaprejšnjih določenih funkcij, ki jih lahko uporabimo za analizo. Nekatere izmed teh funkcij vključujejo: **sin()** (sinus), **cos()** (kosinus), **tan()** (tangens), **exp()** (eksponentna funkcija), **ln()** (naravni logaritem) in še več. (celoten seznam (v ang.) tukaj) Te funkcije lahko uporabimo za ustvarjanje grafov in izvajanje različnih analiz.

Za uporaba funkcije v GeoGebri, preprosto vnesite ime funkcije, sledeno z oklepaji, ki vsebujejo argumente funkcije. Na primer, za risanje grafa funkcije $f(x) = \sin(x)$, vnesite **f(x) = sin(x)** v vhodno vrstico in pritisnite .

Lahko tudi ustvarimo funkcije, ki so odvisne od drugih objektov v GeoGebri. Na primer, z ukazom **g(x) = ax³ + bx² + cx + d** lahko ustvarimo funkcijo, ki je odvisna od spremenljivk **a, b, c** in **d**. Te spremenljivke lahko nato nastavimo na določene vrednosti ali jih povežemo z drsniki za dinamično prilagajanje funkcije.

Nekatera pomembna orodja za delo s funkcijami so:

- Ukaz **Roots(<funkcija>, <min x>, <max x>)** najde ničle funkcije na intervalu $[\min x, \max x]$.
- Ukaz **Max(<funkcija>, <min x>, <max x>)** najde maksimum funkcije na določenem intervalu.
- Ukaz **Min(<funkcija>, <min x>, <max x>)** najde minimum funkcije na intervalu $[\min x, \max x]$.

Če hitro želimo najti nekatere lastnosti funkcije, lahko uporabimo orodje 

Function Inspector (Lastnosti funkcije). To orodje nam omogoča hitro analizo funkcije in izračun njenih lastnosti, kot so ničle, ekstremi, infleksijske točke in še več.

Ko delamo s funkcijami v LaTeXu, je lahko koristno, da uredimo funkcije, s katerimi delamo. Za to, lahko uporabimo orodja 

Input box (Vhodno polje). Ko ustvarimo vhodno polje, dobimo okno kjer določamo objekt, ki ga želimo prikazati in urejati *Linked Object (Povezani objekt)* in *Caption (Naslov)*, ki bo prikazan poleg vhodnega polja.

Odводи in integrali

Polarni krivulje

GeoGebra omogoča tudi risanje in analizo polarnih krivulj. Polarne krivulje so definirane z enačbami v polarnih koordinatah, kjer je vsaka točka določena z razdaljo od izhodišča (polmer) in kotom glede na pozitivno smer osi x .

Poleg tega, lahko prikažemo polarno omrežje v grafičnem oknu z klikom na .

Vrstica za zamenjavo sloga, nato na .

Show grid (Omrežje) in izberemo možnost .

Polar Grid (Polarno omrežje).

Parametrične krivulje

Polarne krivulje so posebna vrsta parametričnih krivulj, kjer je polmer funkcija kota. Vendar pa lahko ustvarimo tudi splošne parametrične krivulje v GeoGebri z uporabo ukaza **Curve**. Ta ukaz omogoča definiranje krivulje z uporabo dveh funkcij, ki določata x in y koordinate glede na parameter t . Na primer, ukaz `Curve((cos(t), sin(t)), t, 0, 2*π)` ustvari enoto krožnico.

Vaje

- Exercise ?? Ustvarite Peaucellier-Lipkinov mehanizem v GeoGebri in analizirajte sled točke F .
- Exercise ?? Kako se premika mačka, ko se lestev zdrsne po tleh?
- Exercise ?? Ustvarite funkcijo in analizirajte njene lastnosti z orodji GeoGebre.
- Exercise ?? Ustvarite konstrukcijo v GeoGebri, ki omogoča prikaz definicije odvoda funkcije.
- Exercise ?? V konstrukciji iz prejšnje vaje, dodajte možnost za prikaz grafa odvoda funkcije.
- Exercise ?? Ustvarite konstrukcijo v GeoGebri, ki omogoča prikaz določenega integrala funkcije med točkama a in b .
- Exercise ?? Ustvarite polarne krivulje in analizirajte njihove lastnosti z orodji GeoGebre.

0.2.4 Tridimenzionalna geometrija z GeoGebro

GeoGebra 3D omogoča delo s tridimenzionalnimi objekti in njihovo vizualizacijo. V tem poglavju bomo raziskali osnovne funkcije GeoGebre 3D ter kako jih uporabiti za ustvarjanje in analizo tridimenzionalnih geometrijskih oblik.

Za delo v GeoGebri 3D moramo najprej preklopiti na 3D pogled. To storimo tako, da iz menija izberemo “Perspectives” in nato “3D Graphics”. Prikaz “3D Graphics” vključuje tridimenzionalno koordinatno mrežo, ki nam omogoča ustvarjanje in manipulacijo 3D objektov, in tudi algebrsko okno, kjer lahko vidimo seznam vseh ustvarjenih objektov in njihovih lastnosti, podoben kot v 2D pogledu.

Točke v 3D prostoru ustvarimo z orodjem .

Point (Točka). Kliknemo na orodje in nato v 3D prostoru izberemo lokacijo, kjer želimo postaviti točko. Za to, moramo najprej določiti koordinate x in y v ravnini (sivi ravnini na 3D prikazu), nato pa določimo višino z tako, da povlečemo točko navzgor ali navzdol. Ko ustvarimo točko, se njene koordinate prikažejo v algebrskem oknu. Alternativno lahko točko ustvarimo tudi z vnosom njenih koordinat v algebrsko okno, na primer $A = (2, 3, 4)$.

Podobno, za premikanje točk v 3D prostoru uporabimo orodje .

Move (Premakni). Nato lahko preklapljate med dvema načinoma s ponavljajočim klikom na točko, medtem ko je orodje aktivno: premikanje po ravnini (vzporedno v sivo ravnino) in premikanje navpično (v smeri osi z).

Najlažje način za učenje orodje je, da jih raziskujete.

Točke in krogle

Včasih je težko vse videti v 3D prostoru. Priporočamo, da eksperimentirate z različnimi pogledi in orodji za navigacijo v GeoGebri 3D. Vse najdete v .

Styling bar (Vrstici za spremenite sloga pogleda). Če želite ročno spremeniti pogled, uporabite orodje .

Rotate View (Zasukaj pogled) ali orodje .

Translate View (Premakni pogled).

Premice in ravnine

Poliedri in drugi 3D objekti

Rotacijska ploskev

Drugi način za prikaz rotacijske ploskve je z orodjem .

Rotate around Line (Vrtenje okoli premice).

Funkcije z več spremenljivkami.

Vaje

- Exercise ??: Ustvarite dve krogli v GeoGebri 3D in raziskujte njihovo presečišče.
- Exercise ??: Kako izmeriti razdaljo medo točkama v GeoGebri 3D?
- Exercise ??: Ustvarite kocko in ravnino v GeoGebri 3D ter poiščite njihovo presečišče.
- Exercise ??: Ustvarite različne poliedre in 3D oblike v GeoGebri 3D.
- Exercise ??: Ustvarite rotacijske ploskve v GeoGebri 3D z vrtenjem daljic in krožnic okoli osi.
- Exercise ??: Ustvarite rotacijske ploskve z vrtenjem grafov funkcij okoli osi v GeoGebri 3D.
- Exercise ??: Raziščujte funkcije z več spremenljivkami v GeoGebri 3D.

0.2.5 GeoGebra: Izvoz in ustvarjanje učnih gradiv

V izdelavi.

0.3 SageMath

0.3.1 SageMath: Uvod

Kaj je SageMath (SageMath)?

- SageMath (na kratko Sage) je brezplačen odprtokodni programski sistem za matematiko.
- Lahko si ga predstavljate kot Phyton na matematičnih steroidih.
- Poslanstvo sistema SageMath je ustvariti učinkovito brezplačno odprtokodno alternativo sistemom Magma, Maple, Mathematica in Matlab.

Glej tudi

SageMath se pogosto uporablja in na spletu je na voljo veliko gradiva. Navajam nekaj najuporabnejših virov.

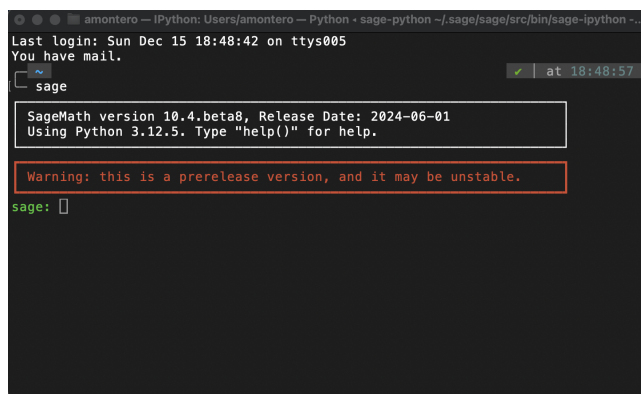
- SageMath Documentation: <https://doc.sagemath.org/>
- SageMath Reference Manual: <https://doc.sagemath.org/html/en/reference/index.html>
- A tour of SageMath: https://doc.sagemath.org/html/en/a_tour_of_sage/
- Introductory SageMath tutorial: <https://doc.sagemath.org/html/en/prep/Intro-Tutorial.html>
- SageMath tutorial: <https://doc.sagemath.org/html/en/tutorial/index.html>
- SageMath thematic tutorial: https://doc.sagemath.org/html/en/thematic_tutorials/index.html
- SageMath repository of interacts (interaktivni programčki): <https://wiki.sagemath.org/interact>
- SageMath quickref cards (reference za pomoč): <https://wiki.sagemath.org/quickref>

Kako uporabljamo SageMath? SageMath lahko uporabljate na več načinov, vsak od njih ponuja različno raven nadzora in različne funkcije. Glavni načini uporabe so:

- **SageCell** <https://sagecell.sagemath.org/> Zelo uporabno za enostavne in hitre naloge.
- **Lokalna namestitvev.** Ni priporočljivo za začetnike. Če nimate izkušenj z nameščanjem programske opreme, je to lahko nekoliko zapleteno. Poleg tega skoraj nikoli ni potrebna. Edini dober razlog za lokalno namestitev programa SageMath je, da nimate dostopa do interneta.
- **Interaktivni programčki.** Podobno kot apleti GeoGebre. Za uporabo interaktivnih programčkov ni treba imeti predznanja. Za njihovo ustvarjanje pa potrebujete napredno poznavanje programa SageMath, ki presega vsebine tega predmeta.
- **CoCalc.com** <https://cocalc.com/>. S preprostimi besedami, CoCal je za SageMath, kar je Overleaf za LaTeX.

SageMath na CoCalcu Obstajajo različne načine za uporabo SageMath, najpogostejše so:

- Na ukazni vrstici (terminalu). Ta način je primeren za napredne uporabnike in ni priporočljiv za začetnike. V tem primeru, SageMath deluje podobno kot običajen programski jezik, ukaze napišemo v terminal in dobimo rezultate. Ukazna vrstica običajno izgleda kot Figure 5.



```

Last login: Sun Dec 15 18:48:42 on ttys005
You have mail.

sage

SageMath version 10.4.beta8, Release Date: 2024-06-01
Using Python 3.12.5. Type "help()" for help.

Warning: this is a prerelease version, and it may be unstable.

sage: 
```

Figure 5: Ukazna vrstica SageMath

- **Scripting:** Kodo napišemo v besedilno datoteko in jo nato zaženete v ukazni vrstici. Ta način je prav tako primeren za napredne uporabnike in ni priporočljiv za začetnike, saj zahteva poznavanje osnov programiranja.
- **Interaktivno v zvezku (Jupyter Notebook).** Ta način je najbolj primeren za začetnike in je tisti, ki ga bomo uporabljali v tem predmetu. SageMath lahko uporabljamo v zvezkih Jupyter, ki omogočajo kombiniranje besedila, matematičnih izrazov in kode v enem dokumentu. Zvezki so zelo uporabni za učenje, raziskovanje in dokumentiranje matematičnih izračunov. V tem predmetu bomo uporabljali CoCalc, spletno platformo, ki omogoča uporabo SageMath v zvezkih Jupyter brez potrebe po lokalni namestitvi.

V beležnici Jupyter imamo dve vrsti celic: *Code* (koda) in *Text* (besedilo) (glej Figure 6).

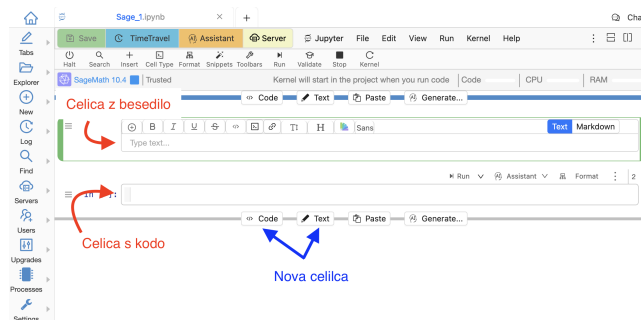


Figure 6: Celice v Jupyter

V celici z besedilo lahko uporabite *Markdown* ali standardne besedilo. Priporočamo, da uporabite *Markdown*, saj omogoča boljše oblikovanje besedila in vključevanje matematičnih izrazov.

Glej tudi

Markdown je preprost jezik za označevanje besedila, ki omogoča enostavno oblikovanje besedila. Pristop je podoben LaTeX-u, vendar je veliko enostavnejši za uporabo, na žalost, pa tudi manj zmogljiv.

Osnovne značilnosti Markdowna vključujejo:

- **Naslovi:** Uporabite **# Naslov** za naslove. Več # pomeni nižjo raven naslova.
- **Krepko in ležeče besedilo:** Uporabite ****krepko**** za **krepko** besedilo in *_ležeče_* za *ležeče besedilo*.
- **Seznami:** Uporabite **-** ali ***** za neurejene sezname in številke za urejene sezname.
- **Povezave:** Uporabite **[besedilo] (URL)** za ustvarjanje povezav.
- V Jupyter zvezkih lahko vključite tudi matematični izraze z uporabo LaTeX sintakse, na primer $x^2 + y^2 = z^2$ za inline izraze in $E = mc^2$ za prikazne izraze.

Za več informacij o Markdownu si oglejte naslednje vire: Markdown osnovna sintaksa
Ta učbenik je napisan v *Myst Markdown*, ki je razširitev Markdowna.

V celici z kodo vnesemo ukaze SageMath. Za izvedbo kode pritisnite **+** ali kliknite na gumb “Run” v orodni vrstici. Rezultati se bodo prikazali neposredno pod celico z kodo.

SageMath kot kalkulator SageMath lahko uporabljamo kot kalkulator za osnovne aritmetične operacije, kot so seštevanje, odštevanje, množenje in deljenje. Poleg tega lahko uporabljamo tudi bolj zapletene matematične funkcije in operacije.

Z nekaj manjšimi izjemami Sage uporablja programski jezik Python, zato so osnovne aritmetične operacije in druge osnove programiranja v SageMath zelo podobne tistim v Pythonu.

Poglejte naslednje primere. Še bolj pa jih preizkusite sami v svojem CoCalc zvezku!

Table 8: Osnovne aritmetične operacije v SageMath-u

Seštevanje ($a + b$)	<code>a+b</code>
Odštevanje ($a - b$)	<code>a-b</code>
Množenje (ab)	<code>a*b</code>
Deljenje ($\frac{a}{b}$)	<code>a/b</code>
Potenciranje (a^b)	<code>a**b</code> ali <code>a^b</code>
Kvadratni koren ($\sqrt{a}$)	<code>sqr(a)</code>
Splošni koren ($\sqrt[n]{a}$)	<code>a^(1/n)</code>

`1+1`

`( 1 + 2 * (3 + 5) ) * 2`

Simbol `*` pomeni množenje, ki se ne sme izpustiti, tudi v izrazih, kot je $2x$: `2*x`. Za združevanje simbolov uporabljamo oklepaj `( )` nikoli `[ ]` ali `{ }`.

`2**3` # `**` pomeni potenciranje, kot v Pythonu

`8`

`2^3` # ampak `^` pomeni tudi potenciranje, za razliko od Pythona

`-3^2` # pazite!

`-9`

Vrstni red izvajanja operacij je določen v *Arithmetical binary operator precedence*.

`20/6` # Deljenje se označi z `/`

Pri vzeto SageMath poskuša privzeto vrniti izraz, ki je čim bolj natančen. To je $10/3$ vrne racionalno število $10/3$ (in ne decimalno približek, kot je $3,33333$). Če želimo številčni približek, lahko enemu od števil preprosto dodamo decimalno piko.

`20.0/6`

Kvadratne korene lahko izračunamo z ukazom `sqr(a)`. Upoštevajte, da če je argument celo število, SageMath interpretira rezultat kot simbolno število.

`sqr(2)`

`sqr(2)`

`sqr(8)`

Seveda, lahko dobimo številčni približek z ukazom `sqr(8.0)`. Na splošno lahko dobimo numerične približke izrazov z (enakovrednimi) ukazi `n()`, `N()`, `numerical_approx()`. Vsi zgornji ukazi imajo neobvezen argument `digits` za določitev natančnosti približke.

`n(sqr(8), digits=50)`

V Table 8 so povzete osnovne aritmetične operacije v SageMath-u.

Operacije s celimi števili S simboli `//` in `%` dobimo (celoštevilski) kvocient oziroma ostanek dveh celih števil

$$20 = 6 \cdot 3 + 2 \quad (2)$$

`20 // 6`

`20 % 6`

`divmod(20,6)` # vrne kvocient in ostanek hkrati kot tuple

Izračunamo lahko tudi fakultete in binomske koeficiente

$$3! = 6 \quad (3)$$

`factorial(3)`

$$\binom{4}{2} = 6 \quad (4)$$

`binomial(4,2)`

Število lahko faktoriziramo kot produkt praštevil

`factor(factorial(7))`

Lahko uporabimo naslednji način tudi

`(factorial(7)).factor()`

`641 * 6700417`

Če želimo vedeti, ali je število praštevilo, lahko uporabimo ukaz `is_prime()`:

`is_prime(29)`

`is_prime(65537)`

Delitelje števila lahko dobimo na naslednji način.

`12.divisors()`

Če samo želimo delitelje, ki so praštevila:

`120.prime_divisors()`

`120.prime_factors()`

Izračunamo lahko *največji skupni delitelj* in *najmanjši skupni večkratnik* dveh števil.

$$D(120, 48) = \quad (5)$$

`gcd(120,48)`

$$v(9, 15) = \quad (6)$$

`lcm(9,15)`

Matematične konstante in funkcije Pogoste konstante in funkcije so določene

$$\sin(\pi) = 0 \quad (7)$$

`sin(pi)`

`cos(pi/4)`

`\(\displaystyle \frac{1}{2} \, \, \sqrt{2}\)`

Vsak objekt v SageMath-u ima predstavitev LaTeX, ki jo dobimo z ukazom `latex()`

`latex(cos(pi/4))`

`\(\displaystyle \frac{1}{2} \, \, \sqrt{2}\)`

Če želimo da izhod je predstavljen v LaTeX-u, uporabimo čarobni ukaz `%display latex`

`%display latex #` Ponovno zaženite zgornje primere, potem ko izvedete ta ukaz.

Če želimo vrniti standardni izhod, uporabimo ukaz:

`%display plain`

V SageMath lahko delamo tudi z kompleksnimi števili. Imaginarno število i je označeno z `i` in z `I`. Ker pa se `i` pogosto uporablja pri programiranju, je dobra praksa, da se označi z `I`.

`arg(1+I)`

`latex(1+I)`

`i + 1`

Nekateri viri namesto e raje pišejo `exp` in SageMath meni, da je to v redu.

`e^3 - exp(3)`

Ukaza `max()` in `min()` vrnete največje in najmanjše število v nizu števil.

`max(1,2,3)`

`min(1,2,3)`

V SageMathu, podobno kot v Pythonu, so sezname zapisani v oklepajih `[]`

`max([2,3,4,5])`

Ukaz `floor()` zaokroži število navzdol na najbližje celo število, ukaz `ceil()` pa zaokroži navzgor.

`floor(2.1)`

`ceil(2.1)`

Pazite!

`floor(1/(2.1 - 2))`

Ampak

$$\left\lfloor \frac{1}{2.1 - 2} \right\rfloor = \left\lfloor \frac{1}{.1} \right\rfloor = \lfloor 10 \rfloor = 10 \quad (8)$$

Kaj se je zgodilo?

Računalniki shranjujejo realna števila v binarni obliki, mi pa smo navajeni uporabljati desetiško predstavitev. Število 2.1 v desetiškem zapisu je precej preprosto in kratko, vendar je po pretvorbi v binarni zapis $10.000\overline{11} = 10.0001100110011\dots$

Ker računalniki ne morejo shraniti neskončnega števila števil, se to število nekje zaokroži, kar povzroči manjšo napako, ki smo jo videli. V SageMathu pa so racionalna števila (ulomki) natančna, zato te napake pri zaokroževanju nikoli ne bomo videli.

`floor(1/ ( (21/10) - 2) )`

Pripisovanje, enakost, enačbe in neenačbe.

SageMath uporablja `=` za pripisovanje. Za primerjavo uporablja `==`, `<=`, `>=`, `<` in `>`:

```
a=5
```

```
a
```

```
2 == 2
```

```
2<=3
```

```
2<2
```

```
a < 3
```

```
a == 5
```

Če želimo shraniti rezultat izračuna, ga dodelimo spremenljivki:

```
y=1+2
```

Pazite: Pogosta napaka je uporaba izraza `a=b` za primerjavo `a` in `b`, vendar na koncu vrednost `a` zamenjate z vrednostjo `b`.

```
y*(y+2)
```

Rezultat izračuna se ne natisne samodejno, ko je pripisan spremenljivki. Zato bomo naredili naslednje, da ga tudi natisnemo,

```
b=3*4 ; b
```

Znak `;`, ki ločuje več navodil v isti vrstici. Če ni določeno drugače, se v beležnici Jupyter natisne samo izpis, ki ga ustvari zadnja vrstica. Zato je uporaba znaka `;` enakovredna uporabi nove vrstice v isti celici.

```
y = 3 + 2; y
```

```
y+2
```

Če želimo natisniti vsak korak, moramo uporabiti ukaz `print()`

```
y = 1 + 2; print(y)
```

```
y = 3 * y + 1; print(y)
```

```
y = 3 * y + 1; print(y)
```

```
y = 3 * y + 1; print(y)
```

- Če znate programirati v Pythonu, boste morda ugotovili, da je to standardno obnašanje spremenljivke.
- Ni priporočljivo na novo definirati vnaprej določenih konstant in funkcij iz programa Sage.
- Čeprav to ne vpliva na notranje obnašanje programa Sage, lahko privede do presenetljivih rezultatov:

```
pi = -I/2
```

```
exp(2*I*pi)
```

Ukaz `restore()` povrne privzeto vrednost vsem vnaprej določenim spremenljivkam in funkcijam.

```
restore()
```

`y+pi`

Ukaz `reset()` opravi še popolnejšo ponastavitev, zlasti izbriše vse uporabniško definirane spremenljivke.

`reset()`

`y`

Opazite tudi, da SageMath včasih razlikuje med simbolnimi izračuni in števili.

`e^(2*pi*I)`

`e^(2*pi*I) == 1`

SageMath zgornjega ne razume kot primerjavo, temveč kot enačbo.

`3*x - 10 == 5`

Enačbo ali neenakost rešimo z ukazom `solve()`.

`solve(3*x - 2 == 5, x)`

Privzeto SageMath interpretira `x` kot simbolno spremenljivko. Za zdaj bomo reševali samo `x`, pozneje pa bomo videli, kako določiti lastne (simbolne) spremenljivke

`solve( 2*x - 5 >= 17, x)`

`solve( x^2 + x == 6, x)`

`solve(2*x^2 - x + 1 == 0, x)`

Množica rešitev nekaterih neenačb je sestavljena iz unije in preseka odprtih intervalov.

`solve( x^2 - 6 >= 3, x )`

`solve( x^2 - 6 <= 3, x )`

- Ukaz `solve()` poskuša izraziti rešitev enačbe brez uporabe števil s plavajočo vejico.
- Če tega ni mogoče storiti, bo vrnil rešitev v simbolični obliki.

`solve( sin(x) == x, x)`

- Za iskanje numeričnega približka rešitve lahko uporabimo ukaz `find_root()`.
- Ta ukaz zahteva izraz in zaprt interval, na katerem se išče rešitev

`find_root(sin(x) == x, -pi/2 , pi/2)`

- Ob začetku seje SageMath ustvari eno simbolno spremenljivko, `x`, ki se lahko uporablja za reševanje enačb.
- Če želite uporabiti dodatno simbolno spremenljivko, jo morate deklarirati z ukazom `var()`.

`y,z,t = var("y z t")`

`phi, theta, rho = var("phi theta rho")`

`x1, x2 = var("x1 x2")`

- Imena spremenljivk ne smejo vsebovati presledkov
- Na primer `square root` ni veljavno ime spremenljivke, `square_root` pa je.

Poskus uporabe simbolne spremenljivke, preden je bila deklarirana, bo povzročil napako `NameError`.

```
u
```

```
solve (u^2 -1,u)
```

Simbolno spremenljivko, kot je zgoraj opredeljena spremenljivka `phi`, lahko razgllašamo z ukazom `restore()`.

```
restore('phi')
```

```
phi
```

Majhne sisteme linearnih enačb lahko rešite tudi s ukazom `solve()`, če imajo vse simbolne spremenljivke so bile deklarirane.

```
x,y,z = var('x','y','z')
```

```
solve( [3*x - y == 2, -2*x -y == 1 ], x,y)
```

```
[[x == (1/5), y == ( -7/5)]]
```

```
solve( [ 2*x - y == -1 , 2*x - y == 2],x,y)
```

```
solve( [ 2*x + y == -1 , -4*x - 2*y == 2],x,y)
```

- `r1` označuje, da obstaja prosta spremenljivka, ki določa množico rešitev.
- Če je več prostih spremenljivk, jih SageMath našteje `r1, r2, ..., rk`.

```
solve([ 2*x + 3*y + 5*z == 1, 4*x + 6*y + 10*z == 2, 6*x + 9*y + 15*z == 3], x,y,z)
```

```
[[x == -5/2*r3 - 3/2*r4 + 1/2, y == r4, z == r3]]
```

- Ukaz `solve()` je lahko pri velikih sistemih enačb zelo počasen.
- Za te sisteme je najbolje uporabiti funkcije linearne algebre, saj so precej učinkovite. Kmalu se bomo naučili, kako.

Reševanje neenačb v več spremenljivkah lahko privede do zapletenih izrazov, saj so območja, ki jih določajo, zapletena.

```
solve([ x -y >=2, x+y <=3], x,y)
```

Vaje

1. Poiščite vse rešitve enačbe $x^3 - x = 7x^2 - 7$
2. Poišči celotno množico rešitev za neenačbo $|t - 7| \geq 3$
3. Poiščite vsa x in y , ki izpolnjujejo tako $2x + y = 17$ kot $x - 3y = -16$.
4. S funkcijo `find_root()` poiščite rešitev enačbe $e^x = \cos(x)$ na intervalu $[-\pi/2, 0]$.
5. Zgornji ukaz spremenite tako, da `find_root()` najde drugo rešitev na istem intervalu.

Lepe praznike!

```
var('t k')
from sage.plot.plot3d.shapes import *
num_wraps=4
num_lights=30
c=rainbow(num_lights)
f(t) = ( (1 -t)*cos(2*pi*t*num_wraps), (1 -t)*sin(2*pi*t*num_wraps),t)
lights=add([point(f(1/(num_lights)*(k+1)),color=c[k],size=50) for k in range(0,num_lights)])
S = Cone(1, 1, color='green')
S += parametric_plot3d(f(t),(t,0,1),thickness=3 )
S += lights
S.show(frame=false)
```

0.3.2 Algebra s SageMath

V izdelavi.

0.3.3 Sage: Analiza in risanje grafov.

V izdelavi.

0.4 Priloge

0.4.1 Nameščanje LaTeXa na svoj računalnik

LaTeX je popolnoma brezplačen in ga lahko enostavno namestite na svoj računalnik. Uradno spletno stran za namestitev si lahko ogledate tukaj (v angleščini).

Za uporabo LaTeXa potrebujete v bistvu dve orodji:

- distribucijo LaTeX, ki je sam program LaTeX, s katerim boste ustvarjali svoje dokumente.
- urejevalnik besedil, ki je vmesnik, ki ga boste uporabljali za pisanje in izdelavo dokumentov. Verjetno že imate urejevalnik besedil na svojem računalniku (npr. Notepad ali TextEdit). Ti urejevalniki so večnamenski in imajo običajno zelo malo orodij. Na tej strani priporočamo nekaj možnosti, ki vam bodo olajšale delo.

Windows

Namestitev distribucije

1. Odprite spletno stran MiKTeX.
2. Prenesite namestitveni program za Windows.
3. Zaženite namestitev in sledite navodilom (pustite privzete nastavitve).
4. Po zaključku preverite namestitev tako, da v ukazni vrstici (Command Prompt) vpišete:

```
pdflatex - -version
```

Priporočeni urejevalniki

- Visual Studio Code (potrebujete tudi razširitev *LaTeX Workshop*). Še bolje, predlagamo, da namestite VSCodium, brezplačno/odprtokodno različico vscode.
- TeXworks (preprost, pogosto nameščen z MiKTeX).
- TeXstudio (bolj napreden, uporabniku prijazen).

macOS

Namestitev distribucije

1. Odprite spletno stran MacTeX.
2. Prenesite paket **MacTeX.pkg** (velik ~4 GB).
3. Odprite preneseno datoteko in sledite navodilom za namestitev.
4. Preverite namestitev v Terminalu:

```
pdflatex - -version
```

Priporočeni urejevalniki

- Visual Studio Code (razširitev *LaTeX Workshop*). Še bolje, predlagamo, da namestite VSCodium, brezplačno/odprtokodno različico vscode.
- TeXShop (priložen MacTeX-u).
- TeXstudio.

Linux (primer: Ubuntu/Debian)

Če uporabljate Linux, verjetno ne potrebujete naših navodil.

Namestitev distribucije

1. Odprite Terminal.
2. Namestite TeX Live:

```
sudo apt update  
sudo apt install texlive -full
```

3. Preverite namestitev:

```
pdflatex - -version
```

Priporočeni urejevalniki

- Visual Studio Code. (razširitev *LaTeX Workshop*). Še bolje, predlagamo, da namestite VS-Codium, brezplačno/odprtokodno različico vscode.
- Kile (posebej priljubljen v Linux okolju).
- TeXstudio.
- TeXworks.

0.4.2 Izvoz datotek iz Overleaf projekta

Če želite izvoziti celoten projekt iz Overleafa kot **.zip** ali **pdf** datoteko, sledite naslednjim korakom:

1. **Odprite projekt** v Overleafu, iz katerega želite prenesti datoteke.
2. V zgornjem levem kotu kliknite na gumb **Menu**.
3. V spustnem meniju izberite možnost **Download >Source** (če želite **.zip** datoteko) ali **Download >PDF** (če želite **.pdf** dokument).
4. Overleaf bo ustvaril in prenesel **.zip** datoteko, ki vsebuje vse datoteke projekta (npr. **.tex** datoteke, slike, **.bib** datoteke, ipd.) oziroma **.pdf** dokument.
5. Shrani preneseno datoteko na svoj računalnik in jo po potrebi razširi.

0.4.3 Dolžine v LaTeXu

Dolžine (*lengths*) so pomemben del LaTeXa, saj omogočajo natančen nadzor nad razmikom, velikostjo elementov, zamiki, robovi in še več. Vsaka dolžina je shranjena kot spremenljivka, ki ji lahko dodelimo vrednost, jo povečamo ali zmanjšamo, in uporabimo pri različnih ukazih.

Osnovne merske enote

V LaTeXu lahko dolžine izrazimo z različnimi enotami. Najpogosteje uporabljene so:

Enota	Opis
pt	točka (1/72,27 palca) – osnovna enota v TeXu
mm	milimeter
cm	centimeter
in	palec (2.54 cm)
ex	višina male črke x v trenutni pisavi
em	širina črke M v trenutni pisavi
bp	velika točka (1/72 palca)
pc	pica = 12 točk
dd	didotova točka
cc	cicer = 12 didotov
sp	najmanjša notranja enota v TeXu (65536 sp = 1 pt)

Privzete dolžinske spremenljivke

LaTeX ima več že vnaprej določenih dolžin, ki vplivajo na izgled dokumenta:

Dolžina	Opis
<code>\textwidth</code>	širina glavnega besedila
<code>\textheight</code>	višina glavnega besedila
<code>\linewidth</code>	širina vrstice znotraj okolja (npr. v <code>minipage</code>)
<code>\columnwidth</code>	širina stolpca v dvostolpčnem načinu
<code>\paperwidth, \paperheight</code>	dimenzije papirja
<code>\parindent</code>	zamik prve vrstice odstavka
<code>\parskip</code>	razmik med odstavki
<code>\baselineskip</code>	navpična razdalja med vrsticami besedila
<code>\topmargin, \oddsidemargin,</code> <code>\evensidemargin</code>	robovi strani